

Self-adapting control parameters in particle swarm optimization

Mewael Isiet^{a,*}, Mohamed Gadala^{b,c}

^a Norman B. Keevil Institute of Mining Engineering, 511-6350 Stores Road, Vancouver, BC V6T 1Z4, Canada

^b Mechanical Engineering Department, University of British Columbia, 2054-6250 Applied Science Lane, Vancouver, BC V6T 1Z4, Canada

^c Mechanical Engineering Department, Abu Dhabi University, Abu Dhabi, United Arab Emirates

HIGHLIGHTS

- Developed a novel mechanism to self-adapt the Particle Swarm Optimization method.
- Proposed approach adapts the control parameters of particles and handles constraints.
- Solved a set of optimization problems to study performance and convergence.
- Provided a design study to demonstrate the effectiveness and efficiency of the developed approach.
- Displayed the superiority of the developed mechanism by comparing with some metaheuristic methods.

ARTICLE INFO

Article history:

Received 22 December 2018

Received in revised form 28 June 2019

Accepted 17 July 2019

Available online 29 July 2019

Keywords:

Particle swarm optimization

Metaheuristic algorithm

Self-adaptive mechanism

Constrained optimization

Feedback parameter

ABSTRACT

Particle Swarm Optimization (PSO) is a powerful nature-inspired metaheuristic optimization method. Compared to other methods, PSO can determine the optimal solution in fewer evaluations and generally performs more efficiently and effectively. However, researches show that the PSO method suffers from premature convergence and a dependence on the initial control settings. Due to these shortcomings, the application of PSO may lead to failure in obtaining the global optimal solution. In this work, modifications were performed on the original PSO algorithm to adapt the control parameters to the circumstances of the particles at a specific moment. The proposed method is known as the Unique Adaptive Particle Swarm Optimization (UAPSO). In the developed approach, constraints were handled by forcing the particles to learn from their feasible solutions only. Therefore, the constraint handling technique worked in accord with the adapting scheme to ensure that the particles were adapting to the environment by directing itself to the feasible regions. The performance of UAPSO was verified by a comparative study involving eight benchmark constrained optimization problems and a real-world design problem. The numerical results showed the superiority of UAPSO compared to the selected state-of-the-art metaheuristic methods and PSO variants, its ability in avoiding premature convergence and its consistency and efficiency.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

If a system is defined as a process that consists of inputs that perform certain functions to produce outputs, then by producing a mathematical representation of the system and introducing an evaluation criterion, an optimization problem is created and an optimization method would be required for the solution. Researchers have been developing various algorithms to solve complex and multidimensional problems and depending on factors such as the complexity of the problem, availability of time, computational power and knowledge of the behavior of the functions, an optimization method may be selected [1]. Nature is commonly used for the inspiration in developing optimization algorithms

such as the genetic algorithm (GA) [2], ant colony optimization (ACO) [3], particle swarm optimization (PSO) [3], firefly algorithm [4], moth-flame optimization [5], bat algorithm (BA) [6], chicken swarm optimization (CSO) [7], cuckoo search [8], lion optimization algorithm [9] etc. These algorithms are referred to as Nature-Inspired Algorithms (NIA). Particle Swarm Optimization (PSO) is a stochastic population-based metaheuristic method that takes its inspiration from the social behavior of animals foraging for food such as flocking with birds and schooling with fish [10]. In order to determine which optimization algorithm will perform better across every optimization problem, a statistical approach requiring a very high and unrealistic number of runs would need to be implemented, which is impractical but higher confidence can be placed on algorithms that consistently perform better than others. One of the main strong features of PSO is its use of memory in updating the locations of particles; this factor is

* Corresponding author.

E-mail address: mewael_d@alumni.ubc.ca (M. Isiet).

Table 1
Some state-of-the-art PSO variants.

PSO variant	Description
Constriction model [30]	A constriction coefficient was introduced to balance the local and global search trade-off by damping the vector components in the particle velocity-update equation. With this factor, velocity clamping is not needed and convergence is enhanced.
Fully informed model [31]	In the fully informed model, all particles learn from each other, i.e. particles update their velocity by using the best positions of all members in the swarm.
HPSO-TVAC [32]	A momentum-less PSO is coupled with linearly decreasing inertia weight and time varying acceleration coefficients. To prevent static behavior, a re-initialization strategy is applied to a particle's velocity through a time-based scheme.
CPSO [33]	In CPSO, constrained optimization problems are solved by generating two swarms; one for determining the optimal design variables and the other for finding appropriate penalty factors.
IPSO [34]	IPSO was developed by creating multiple sub-swarms from a population of randomly distributed particles. An individual sub-swarm evolves based on PSO and at a certain predefined time, particles are shuffled and sub-swarms are reassigned. This process ensures that particles are learning new information.
HPSO [35]	In this hybrid algorithm, constraints are handled in PSO by applying feasibility rules. In addition, simulated annealing is applied to the swarm's best obtained solution to help avoid premature convergence.
D-PSO [36]	In an attempt to reduce CPU-time cost, the deterministic PSO (DPSO) algorithm was developed by suppressing the random coefficients (and thus eliminating the stochasticity in PSO) and has been successful in solving single [37] and multi-objective [38] optimization problems.
PSO-DE [39]	In order to solve premature convergence in PSO, the differential evolution (DE) is applied to the best personal solution of each particle obtained within the swarm generated by PSO. Furthermore, constraints are handled by applying a set of feasibility rules.
G-QPSO [40]	In G-QPSO, a particle's trajectory is based on quantum behavior and the swarm is attracted to a local attractor, which is the mean of the personal best solution of each particle. Moreover, a Gaussian probability distribution is employed.
OLPSO [41]	A strategy based on orthogonal learning (OL) was applied to promote the discovery of useful information from the best historical solution of a particle and its neighborhood via orthogonal experimental design.
ALC-PSO [42]	By assigning the swarm leader with a growing age and a lifespan, particles are encouraged to challenge each other for leadership when the leader has aged and thus diversity is promoted.
SL-PSO [43]	Social learning (SL) mechanism is introduced into PSO by making each particle learn from any better performing particle within the current swarm.
EPSO [44]	By selecting a PSO algorithm from an ensemble through a self-adaptive scheme, EPSO solves the optimization problem by ensuring that the best algorithm in producing promising solutions is selected.
A-PSO [45]	Motivated by improving the diversification and searching capabilities of PSO, in A-PSO, the acceleration coefficients and inertia weight varies nonlinearly, and a modified particle position update approach was employed.
PSO-DFCM [46]	A damping factor and cooperative mechanism were incorporated into PSO to enhance performance in large-scale and high-dimensional space, and to prevent premature convergence. The damping factor controls the local exploitation and global exploration of particles, whereas the cooperative mechanism utilizes both the global-best and local-best swarm learning schemes for particle velocity update.
PSO-DLS [47]	In PSO-DLS, the original population is split into multi-sub swarms with each swarm having a local leader. However, to prevent premature convergence, a dynamic learning strategy is implemented whereby particles are classified into the exploitation of their respective sub-swarm or the exploration of undiscovered regions in the design space. The classification mechanism is executed according to a random probability value.

lacking in some NIAs such as GA. When compared to other meta-heuristic methods, the literature shows that PSO proves to be more efficient and robust in obtaining the best solution [11–16]. Also, PSO is a simple and easy-to-implement algorithm with a few control parameters [16]. However, PSO in its original form suffers from a condition that causes it to converge to a local solution prematurely [17] which happens due to the inability of particles to escape from a local minimum and thus stagnate in those regions. Depending on how the control parameters in PSO are set, the performance of the method is affected, and therefore, premature

convergence may occur [18–20]. Furthermore, the PSO method may be ineffective as, at times, it requires an unusually high number of function evaluations to find the optimal solution [21]. This ineffectiveness is further noticeable in multimodal and high dimensional problems [22]. Thus, a need to introduce modifications to the PSO algorithm is present to promote convergence speed and avoid premature convergence. Table 1 contains some state-of-the-art PSO variants. Furthermore, PSO has been modified to solve multi-objective [23,24] and discrete [25,26] optimization problems. Many variations of PSO have been developed either by

merging concepts of PSO with other metaheuristic methods or by introducing novel mechanisms to the PSO algorithm. One such class of PSO variants self-adapts the algorithm through a feedback system — this class is known as the Adaptive Particle Swarm Optimization (APSO) methods. The implemented feedback strategy should be, both, easily computable and effective in directing the particles successfully [27]. Inertia weight and the acceleration coefficients have the most significant impact on the performance of PSO and are thus, typically, selected for adaption since by properly tuning them, PSO could be initiated with a small swarm size or a maximum number of iterations [28]. In a study attempting to understand the behavior of the particles in PSO [29], Kennedy suggested that the sum value of both the cognitive and social component should not exceed four (4). Therefore, it was established since the original component should not exceed four (4). Therefore, it was established since the original conception of PSO to assign a value of 2 for each of the acceleration coefficients [10], whereas the inertia weight was introduced [19] as a means to enhance the performance of the particles by balancing the global and local search. The standard strategy for setting control parameters are based on time by employing either a linear [19] or a nonlinear [48] function. For the particles to navigate the search space effectively, the control parameters would need to be determined based on a feedback state that captures the particles state, instead of following a predefined trajectory [49]. Many real-world engineering design problems are formulated as constrained optimization problems. Presence of constraints has a significant effect on the performance of an optimization algorithm. The original form of any NIA lacks a system for handling constraints [50]. Constraint handling techniques are required to provide necessary information about the feasibility of a specific solution [51]. Therefore, an effective PSO variant should be adaptive and handle constraints. Parsopoulos and Vrahatis [52,53] applied a static penalty function approach which transformed the constrained problem into an unconstrained one by imposing a large penalty whenever a constraint has been violated. In another work, they applied a dynamic penalty function approach where a dynamic penalty value would be employed depending on the degree of constraint violation. Another technique involved the navigation of particles to feasible regions by imposing an initialization scheme that constrained particles to possible solutions only; this method is known as the death penalty [54]. Deb's approach (Feasibility-based rule) is the most commonly applied method due to being simple, parameter-free and flexible since it allows particles to explore infeasible regions [35]. This work proposes a method where particles adapt to the environment by self-directing themselves to feasible locations through the modification of each particle's set of control parameters. The organization of this work is as follows. Section 2 introduces the PSO algorithm and subsequent developments. Section 3 defines the developed self-adaptive method and constraint handling approach. Section 4 compares the performance of the proposed method with other well-known optimizers. Finally, Section 5 concludes the research work.

2. PSO background

In PSO [10], each particle represents a design point in the hyperspace, and its current location is denoted by $\mathbf{x}^{(i,k)}$ where i is the individual particle and k is the iteration number. Each particle keeps track of its current position and its best solution so far denoted by $\mathbf{x}_p^{(i,k)}$. Another value that is tracked by the algorithm is the best global solution denoted by $\mathbf{x}_G^{(k)}$. The particle velocity $\mathbf{v}^{(i,k)}$ refers to the change in particle location at each iteration k [55]. In short, the PSO algorithm attempts to accelerate particles by updating its velocity towards its own personal best and the

global best solution. $\mathbf{v}^{(i,k)}$ and $\mathbf{x}^{(i,k)}$ are both iteratively updated by the following two equations respectively:

$$\mathbf{v}^{(i,k+1)} = \mathbf{v}^{(i,k)} + c_1 \mathbf{rand}_1^{(k)} (\mathbf{x}_p^{(i,k)} - \mathbf{x}^{(i,k)}) + c_2 \mathbf{rand}_2^{(k)} (\mathbf{x}_G^{(k)} - \mathbf{x}^{(i,k)}); i = 1 \text{ to } N \quad (1)$$

$$\mathbf{x}^{(i,k+1)} = \mathbf{x}^{(i,k)} + \mathbf{v}^{(i,k+1)}; i = 1 \text{ to } N \quad (2)$$

where c_1 and c_2 , the cognitive and social parameter respectively, are collectively known as the acceleration coefficients. $\mathbf{rand}_1^{(k)}$ and $\mathbf{rand}_2^{(k)}$ are vectors containing random numbers generated at every k iteration ($0 \leq \mathbf{rand} \leq 1$), and N refers to the swarm size. In the particle velocity-update equation, three vector components exist; namely the momentum vector, the cognitive or personal component and the social or global component. Fig. 1 illustrates how the three velocity vector components influence a particle towards the search for the global best solution. The above two equations represent the original PSO algorithm. Once the particles' new velocity and position have been calculated, $\mathbf{x}_p^{(i,k)}$ and $\mathbf{x}_G^{(k)}$ are updated iteratively with the following equations respectively:

$$\mathbf{x}_p^{(i,k)} = \begin{cases} \mathbf{x}^{(i,k)}, & \text{if } f(\mathbf{x}^{(i,k)}) < f(\mathbf{x}_p^{(i,k)}) \\ \mathbf{x}_p^{(i,k)}, & \text{if } f(\mathbf{x}^{(i,k)}) \geq f(\mathbf{x}_p^{(i,k)}) \end{cases} \quad (3)$$

$$\mathbf{x}_G^{(k)} = \begin{cases} \mathbf{x}^{(i,k)}, & \text{if } f(\mathbf{x}^{(i,k)}) < f(\mathbf{x}_G^{(k)}) \\ \mathbf{x}_G^{(k)}, & \text{if } f(\mathbf{x}^{(i,k)}) \geq f(\mathbf{x}_G^{(k)}) \end{cases} \quad (4)$$

Both the particle's position and velocity are bounded according to a predefined limit. The dynamic range of a particle's position is defined by the design variable limit, whereas the velocity clamping-limit determines the particle's velocity range. In terms of initialization, the particles' positions are randomly distributed across the hyperspace, while their velocities are set as zero [56]. The initial diversity of the swarm profoundly influences the efficiency of the algorithm since the higher the diversity, the higher the chances of them locating the global solution are [57]. Eqs. (1)–(4) are updated at every iteration until a predefined termination criterion is met. Typically, the algorithm terminates once an acceptable global solution is obtained or a predefined maximum number of function evaluations or iterations is reached [58].

2.1. Developments

The introduction of the inertia weight helped promote better searching from particles by balancing the global search and the local search [19]. The inertia weight was incorporated into Eq. (1) as follows:

$$\mathbf{v}^{(i,k+1)} = w \mathbf{v}^{(i,k)} + c_1 \mathbf{rand}_1^{(k)} (\mathbf{x}_p^{(i,k)} - \mathbf{x}^{(i,k)}) + c_2 \mathbf{rand}_2^{(k)} (\mathbf{x}_G^{(k)} - \mathbf{x}^{(i,k)}); i = 1 \text{ to } N \quad (5)$$

where w is an inertia damping term that influences the momentum vector. A large value enhances the particle's global search, and a small value increases the particle's local search. Early on researchers, while comparing the performance of multiple constant inertia weights with a linearly decreasing form, demonstrated a significant improvement in the performance of PSO when the inertia weight was set between a range of 0.9 to 1.2 [19]. In a subsequent study on the linearly decreasing inertia weight (PSO-LDIW) [59], it was reported that varying the inertia weight between 0.9 to 0.4 resulted in improved performance. The linearly decreasing inertia weight equation is described as [60] :

$$w^{(k)} = \left(\frac{(w_{\max} - w_{\min}) \times (\text{Max}_k - k)}{\text{Max}_k} \right) + w_{\min} \quad (6)$$

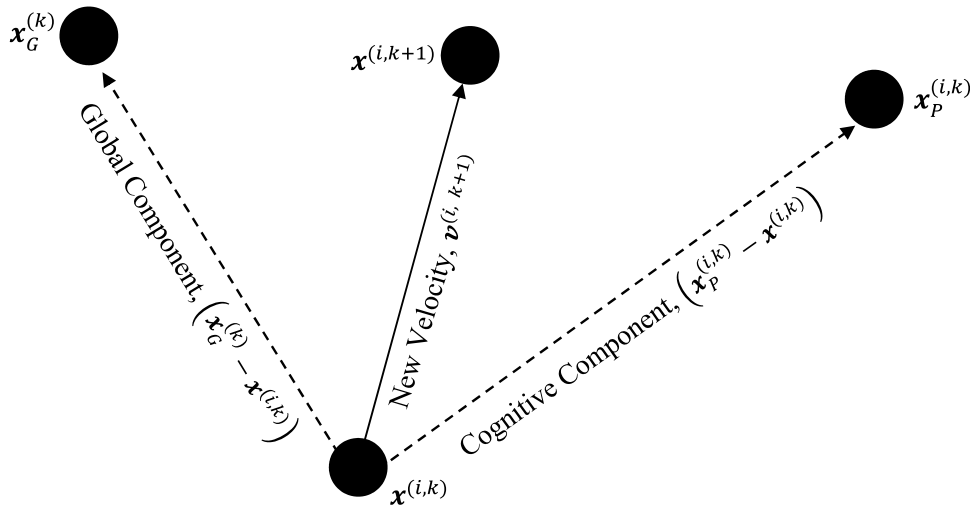


Fig. 1. Particle velocity and position update in a two-dimensional search space.

where w_{min} is the minimum inertia weight value, w_{max} is the maximum inertia weight, k is the current iteration, and Max_k is the maximum iteration. $w^{(k)}$ decreases from w_{max} to w_{min} with respect to k and Max_k . Naturally, different dynamic strategies could be applied to the inertia weight. For example, the random inertia weight (PSO-RIW) by Shi and Eberhart [61] randomly determines the inertia weight at a given iteration according to:

$$\left(w^k = 0.5 + \frac{rand^k}{2} \right) \quad (7)$$

where $rand^k$ is a random number $[0, 1]$, and the range of w^k is between $[0.5, 1]$. It was reported that RIW improved the robustness of the PSO algorithm [61]. Another effective strategy was the use of a fuzzy system to adapt the inertia weight, which proved to be successful in solving dynamic optimization problems [62]. Some researchers utilized previously developed versions of the inertia weight to enhance its influence in the particles' search capability. By introducing an exponential component into the PSO-LDIW, the nonlinear inertia weight (PSO-NLIW) [63] was devised with the following equation:

$$w^{(k)} = \left(\frac{(w_{max} - w_{min}) \times (Max_k - k)^c}{(Max_k)^c} \right) + w_{min} \quad (8)$$

where c is the nonlinear modulation index. It was reported that when $c = 0.9 \sim 1.3$ the inertia weight managed to efficiently navigate the particles between a globalized search in the early iterations to a more refined search in the later iterations [63]. In another adaption, a chaotic inertia weight (PSO-CIW) was proposed by introducing a chaotic mechanism into the PSO-LDIW [48] as follows:

$$w^{(k)} = \left(\frac{(w_{max} - w_{min}) \times (Max_k - k)}{(Max_k)} + w_{min} \right) \times z^k \quad (9)$$

$$z^k = \alpha \times z^{k-1} \times (1 - z^{k-1})$$

where z is the chaotic term and when $3.75 \leq \alpha \leq 4$ the inertia weight behaves chaotically. Moreover, when $\alpha = 4$, the chaotic term covers an interval of $[0, 1]$ if $z^0 \in [0, 1]$ and $z^0 \notin [0, 0.25, 0.5, 0.75, 1.0]$. This strategy promotes more randomness and thus increases the swarm's diversity [64]. Furthermore, empirical results demonstrated that the chaotic inertia weight managed to improve PSO's convergence accuracy, convergence speed, and exploration search ability [48]. An alternative to the inertia weight, the constriction coefficient was developed to balance the local and global search trade-off by damping the vector

components in the particle velocity-update equation. With this factor, velocity clamping is not needed, and convergence is forced. In another study [59], it was empirically shown that by clamping the velocity in the constriction model, quicker convergence was obtained. Besides the inertia weight, the acceleration coefficients play a key role in the overall performance of the algorithm. The motivation for setting $c_1 = c_2 = 2$ was to ensure that on average, a particle would overfly the best historical solutions by half the time [10]. However, this could be harmful to the search for the optimal solution as they control the effectiveness and efficiency of PSO [65]. Therefore the time-varying acceleration coefficients scheme was developed to guide the particles from a global search ($c_1 > c_2$) in the initial stages to a localized search ($c_2 > c_1$) in the final iterations [32]. In another adaption, by using a fuzzy system, the acceleration coefficients varied according to the state of a particle, i.e. a particle search state consisted of exploration, exploitation, convergence and jumping out [49]. Furthermore, Kennedy [66] studied the effect of the acceleration coefficients by comparing a social-only ($c_1 = 0$) and cognition-only ($c_2 = 0$) model of the PSO algorithm. The social-only version managed to outperform the original PSO version and cognition-only model in terms of efficiency and convergence speed [66].

2.2. PSO based on adapting schemes

Many variations of PSO based on adapting strategies exist using different sources as the feedback parameter to track and monitor the condition of the algorithm and make changes according to the circumstances. In one version [67], $w^{(i,k)}$ is adapted based on two feedback parameters called the speed factor and the aggregation degree factor. The speed factor measures whether a particle managed to improve its personal best solution, whereas the aggregation degree factor compares the mean performance of all particles with the best performer in the current iteration. Applying the speed and aggregation factors, $w^{(i,k)}$ is determined as follows:

$$w^{(i,k)} = w_{initial} - \alpha (1 - h^{(i,k)}) + \beta S \quad (10)$$

where $w_{initial}$ is the initial value of the inertia weight, $h^{(i,k)}$ is the speed factor, S is the aggregation factor, α and β are two system components typically set within the range of $[0,1]$. Tanweer et al. [68] developed a scheme where the inertia weight of the best performing particle increased while for the other particles,

it decreased. The self-regulating inertia weight is described as follows:

$$w^{(i,k)} = \begin{cases} w^{(i,k-1)} + \eta \left(\frac{w_{\max} - w_{\min}}{k} \right), & \text{for the best particle} \\ w^{(i,k-1)} - \eta \left(\frac{w_{\max} - w_{\min}}{k} \right), & \text{for all other particles} \end{cases} \quad (11)$$

where $w^{(i,k)}$ is particle i current inertia weight and η is the rate of acceleration. Some researchers opted to use the fitness value of particles in adapting the inertia weight. For example, Arumugam and Rao modified the inertia weight by calculating the ratio of the global best solution to the average value of all personal best solutions at iteration k [69]. Another study [27] proposed an adapting strategy based on the success of particles. If a particle manages to improve its fitness at iteration k , it succeeds otherwise it fails. The inertia weight using the success percentage is computed as follows:

$$w^{(i,k)} = w_{\min} + (w_{\max} - w_{\min}) \left(\frac{\sum_{i=1}^N S^{(i,k)}}{N} \right) \quad (12)$$

where $S^{(i,k)}$ is 1 if successful and 0 if unsuccessful. N represents the swarm size. As the success percentage increases, the inertia weight increases as well and as the success percentage decreases so does the inertia weight. Since these adapting strategies consider some information from the particles, they do not suffer from the limitations of the predefined inertia weight strategies [64]. However, all of the above-mentioned adaptive variants of PSO have been designed to solve unconstrained problems. Also, strategies such as the success percentage [67] and self-regulation [68] have performed poorly when empirically compared to PSO-LDIW [60] and PSO-RIW [61]. This research proposes a method in which a constraint handling technique works in accord with an adapting scheme to ensure that the particles are adapting to the environment by directing itself to the feasible regions.

2.3. Setting control parameters

Poor configuration of the control settings may lead to an increase in the likelihood of premature convergence occurrence [70]. A brief discussion of the control parameters in PSO is given here:

- **Maximum Number of Iterations (Max_k):** If the maximum number of iterations is set as the termination criteria, then the predefined value has a direct impact on the likelihood of the algorithm discovering the global solution. Therefore, when a smaller value is selected, the probability of PSO obtaining the global solution is lower than if a larger value was selected. However, the downside of a large maximum number of iterations is the increased computational cost [60].
- **Swarm Size (N):** When a large swarm size is selected, the particles cover more space in the search area but at the expense of computational efforts. According to Arora [1], anywhere between $5x_N \sim 10x_N$ (where x_N is the number of design variables) is a good estimate, however in some other researches anywhere between $10 \sim 30$ particles [71–73] proved to be successful in solving common optimization benchmark problems.
- **Velocity-Clamping Limit ($V_{\max}$):** The value of the maximum clamping-limit controls the coarseness of the search by restricting the step size (velocity) changes [70]. When a large $V_{\max}$ is set, global exploration is promoted; however, a very large value may cause particles to overfly the global optimum, leading to premature convergence. On the other hand, a smaller $V_{\max}$ facilitates local search, but a very small value may lead to particles requiring more iterations to solve the problem [60,74].

2.4. Proposed constraint handling technique

PSO modifications are needed to handle constrained problems [54]. By the preservation of feasible solutions, particles can search the whole design space but keep track of the possible solutions only, and this can be done through the tracking of constraint violations. In the proposed UAPSO, the following logical conditions are applied for handling constraints:

- **Condition 1:** Any feasible solution is chosen over an infeasible solution.
- **Condition 2:** Between two feasible solutions, the one with the better objective function value is preferred.

By applying the first condition, the algorithm orients itself to favor feasible solutions over infeasible ones, whereas using the second condition forces the algorithm to direct the search towards the feasible regions with better solutions [75]. Fig. 2 shows a schematic of the applied constraint handling technique in UAPSO. As can be seen in Fig. 2(a), $f(\mathbf{x}^{(i,k+1)}) < f(\mathbf{x}^{(i,k)})$ however $\mathbf{x}^{(i,k+1)}$ is not a feasible solution therefore $\mathbf{x}^{(i,k)}$ is retained by the algorithm as the best solution. In Fig. 2(b) since $\mathbf{x}^{(i,k+1)}$ is a feasible solution and $f(\mathbf{x}^{(i,k+1)}) < f(\mathbf{x}^{(i,k)})$, $\mathbf{x}^{(i,k+1)}$ is retained by the algorithm. Reinitialization of particles' initial position until all are within the feasible region was suggested as a method [54] to avoid premature convergence in an infeasible region. However, this approach may increase the computational time. By applying a penalty on unfeasible solutions [76], PSO avoids premature convergence when coupled with the abovementioned conditions. Two pieces of information are included in the penalty function [77] – the number of constraint violations and the amount by which the constraint was violated. The modified PSO algorithm solves the cost function as such:

$$F_i(\mathbf{x}) = \begin{cases} f_i(\mathbf{x}) & , \quad \text{if feasible solution} \\ f_i(\mathbf{x}) + \beta_1 \left(\sum_{i=1}^d h \right) + \beta_2 \left(\sum_{i=1}^d y \right) & , \quad \text{if infeasible solution} \end{cases} \quad (13)$$

where $F_i(\mathbf{x})$ is the penalty function, $f_i(\mathbf{x})$ is the original cost function, $\sum_{i=1}^d h$ is the sum of d violated constraints, $\sum_{i=1}^d y$ is the sum of the amount of d violated constraints and β_1 and β_2 are the penalty factors. Both β_1 and β_2 are set to the value of 10^5 .

3. Unique Adaptive Particle Swarm Optimization (UAPSO)

In this section, we discuss a proposed unique adaptive particle swarm optimization that utilizes a feedback parameter as a mechanism to self-adapt the particles. A unique $c_1^{(i,k)}$, $c_2^{(i,k)}$ and $w^{(i,k)}$ for each particle is the goal of the proposed scheme based on a feedback parameter generated by the fitness of each particle. The proposed self-adaptive method is called the Unique Adaptive Particle Swarm Optimization (UAPSO). In UAPSO, each particle has a unique cognitive parameter $c_1^{(i,k)}$, social parameter $c_2^{(i,k)}$ and inertia weight $w^{(i,k)}$. The evolutionary state ($ES^{(i,k)}$) is the feedback parameter used as the mechanism to self-adapt the particles based on the following:

$$ES^{(i,k)} = \frac{f(\mathbf{x}_p^{(i,k)}) - f(\mathbf{x}_G^{(k)})}{f(\mathbf{x}_f^{(i,k)})} \quad (14)$$

where $f(\mathbf{x}_p^{(i,k)})$ is the fitness of particle i personal best solution, $f(\mathbf{x}_G^{(k)})$ is the fitness of the best historical solution across all

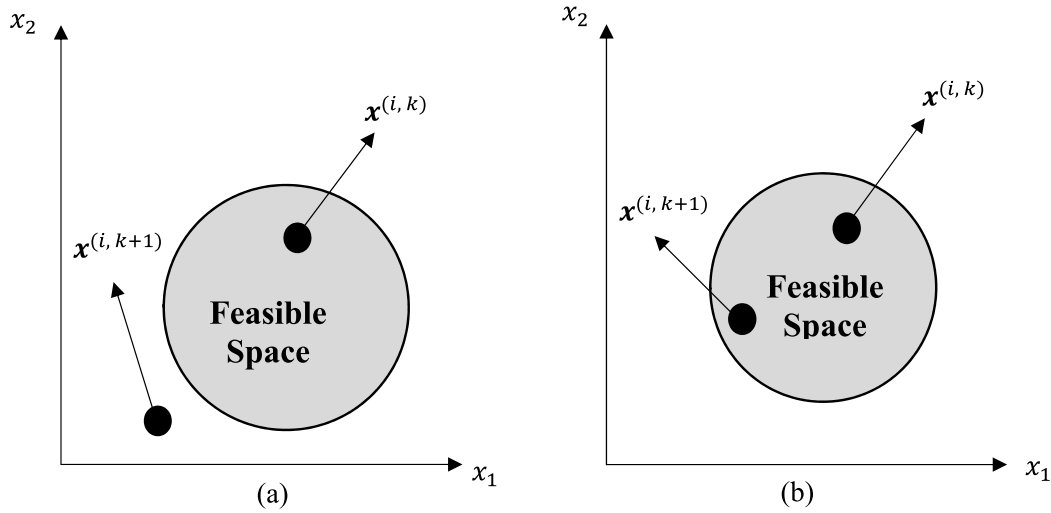


Fig. 2. Schematic diagram of the constraint handling approach with (a) showing preference towards feasibility and (b) showing preference towards feasible solutions with better fitness.

particles and $f(\mathbf{x}_f^{(i,k)})$ is the fitness of each particles latest or most recent feasible solution. Since any given constrained problem involves feasible and infeasible solutions, particles adapting according to the infeasible solutions would result in directing particles towards undesirable locations. In UAPSO particles adopt solely according to the feasible solutions in the design space, this promotes better learning within the swarm. A selection mechanism applied to Eq. (13) enforces the algorithm to retain feasible solutions only. The following equation demonstrates the selection process by which $f(\mathbf{x}_f^{(i,k)})$ is determined:

$$f(\mathbf{x}_f^{(i,k)}) = \begin{cases} f(\mathbf{x}_p^{(i,k)}), & \text{if } k = 1 \\ f(\mathbf{x}^{(i,k)}), & \text{if } \mathbf{x}^{(i,k)} \text{ is a feasible solution} \\ f(\mathbf{x}_f^{(i,k)}), & \text{if } \mathbf{x}^{(i,k)} \text{ is a infeasible solution} \end{cases} \quad (15)$$

In the first iteration, $f(\mathbf{x}_f^{(i,k)}) = f(\mathbf{x}_p^{(i,k)})$ because feasible solutions cannot be guaranteed according to the established constraint handling technique. In the subsequent iterations ($k > 1$), the feasibility of a solution dictates the selection process for the latest feasible solution. Furthermore, due to the feasibility of a particle's best solution and the swarm's best solution, the evolutionary state provides feedback that is reliable as it includes factors that are both feasible and easily computable.

3.1. UAPSO – inertia weight

In the proposed method, the evolutionary state acts as the inertia weight, $w^{(i,k)}$. A large $ES^{(i,k)}$ indicates that a particle's latest feasible solution is near its personal best solution but far from the swarm's best solution, as illustrated in Fig. 3. A small $ES^{(i,k)}$ means either: (1) that the latest feasible solution of the particle is far away from both its personal best and the global leader, shown in Fig. 4(a) or (2) the particle's personal best solution is close to the global leader, illustrated in Fig. 4(b). In cases where $f(\mathbf{x}_f^{(i,k)}) = f(\mathbf{x}_p^{(i,k)})$, a high $ES^{(i,k)}$ means that the particle is far away from the global leader and when $ES^{(i,k)}$ is low, the particle is close to the global leader. Furthermore, whenever $f(\mathbf{x}_p^{(i,k)}) = f(\mathbf{x}_G^{(k)})$, the evolutionary state of the particle is equal to 0. Influence of a particle's momentum is dependent on the value of its inertia weight, the higher the value of $w^{(i,k)}$ the greater the influence of a particle's momentum and the lower the value of $w^{(i,k)}$ the lesser

the influence of a particle's momentum. Thus, at lower $w^{(i,k)}$ particles are more attracted towards their best personal and the swarm's best solution and attempts to converge towards those solutions and at higher $w^{(i,k)}$, particles makes use of their momentum to explore more regions and attempts to discover new optimal solutions. The value of $w^{(i,k)}$ fluctuates between [0, 1].

3.2. UAPSO – acceleration coefficients

Typically and since the conception of PSO [10], the acceleration coefficients has been set as a constant value of 2. However, in subsequent research, a linearly changing version proved to be very successful. Instead of being influenced by time only, in UAPSO a particle's evolutionary state, also, affects the acceleration coefficients. A particle might be more oriented towards either a local or global search and by allowing the evolutionary state to dictate the acceleration coefficient; the algorithm directs the particle to locations with better solutions in the hyperspace, based on its position. In both the conventional way and TVAC, by forcing particles into behaving in a predefined behavior, the search capabilities of the swarm could be affected. The following equations describe the form of both acceleration coefficients applied in the proposed method:

$$c_1^{(i,k)} = \begin{cases} \left(\frac{(c_{max} - c_{min}) \times (Max_k - k)}{Max_k} \right) + c_{min}, & \text{if } 0 \leq ES^{(i,k)} \leq 0.5 \\ c_{max} - \left(\frac{(c_{max} - c_{min}) \times (Max_k - k)}{Max_k} \right), & \text{if } 0.5 \leq ES^{(i,k)} \leq 1.0 \end{cases} \quad (16)$$

$$c_2^{(i,k)} = \begin{cases} c_{max} - \left(\frac{(c_{max} - c_{min}) \times (Max_k - k)}{Max_k} \right), & \text{if } 0 \leq ES^{(i,k)} \leq 0.5 \\ \left(\frac{(c_{max} - c_{min}) \times (Max_k - k)}{Max_k} \right) + c_{min}, & \text{if } 0.5 \leq ES^{(i,k)} \leq 1.0 \end{cases} \quad (17)$$

where $c_{max} = 4$ and $c_{min} = 0$. In this adaption, whenever a particle has a low evolutionary state ($0 \leq ES^{(i,k)} \leq 0.5$), the algorithm promotes a more global search (in the initial iterations) by allowing the particle to explore the regions around its best historical solution and as time goes by a more local search is promoted. In cases where the evolutionary state has a large value ($0.5 \leq ES^{(i,k)} \leq 1.0$), promotion of a more exploitive search, in the initial iterations, allows particles to converge towards the swarm's best solution and gradually, particles perform a global search towards the later stages of the iterations. At larger evolutionary states particles located far from the global best solution have a higher

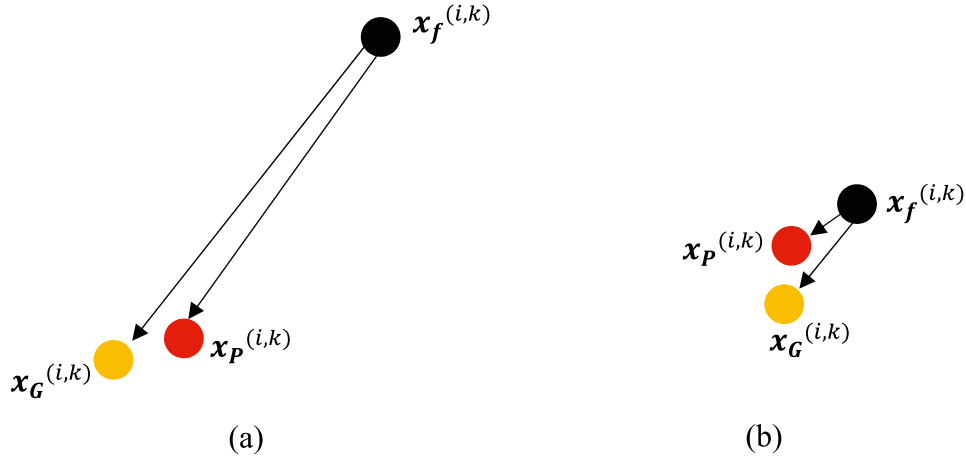


Fig. 3. Schematic description of a small evolutionary state value where a particle's latest feasible solution is (a) far from both historical solutions or (b) close to both historical solutions.

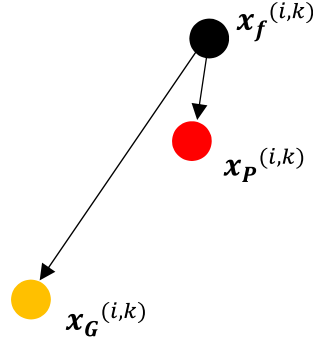


Fig. 4. Schematic description of a large evolutionary state value where a particle's latest feasible solution is in close proximity to its best historical solution but far from the swarm's leader.

momentum and based on its acceleration coefficients, are either attracted to its best historical or the global solution, and this ensures that particles are playing an active role in finding a better solution. At lower evolutionary states, particles explore the design space due to a reduced weight from their respective momentums and the higher influence of either the personal or social component.

3.3. UAPSO – re-initialization of velocity

A particle's momentum is essential to its performance since it provides the necessary acceleration to allow particles to roam around the search space and avoid premature convergence. Hence, a re-initialization scheme is implemented with random velocities whenever stagnation occurs. Stagnation is a phenomenon where particles are unable to improve their solutions due to prematurely converging towards their local solutions [58]. In the proposed method, the re-initialization of a component of the velocity vector with a random velocity occurs whenever stagnation occurs within the swarm. The re-initialization scheme implemented in the proposed method takes its inspiration from the technique applied in the HPSO-TVAC algorithm [32]. Here, only a single component of the velocity vector is reinitialized since the rest of the vector might have a good structure that would enable directing the particle to the global optima or a better local solution. Furthermore, implementation of the re-initialization scheme is according to the velocity clamping-limit

as follows:

$$v_d^{(i, k+1)} = \begin{cases} \text{rand} \times V_{max}^d, & \text{if } v_d^{(i, k+1)} = 0 \text{ and } \text{rand} \leq 0.5 \\ \text{rand} \times (-V_{max}^d), & \text{if } v_d^{(i, k+1)} = 0 \text{ and } \text{rand} > 0.5 \end{cases} \quad (18)$$

where $v_d^{(i, k+1)}$ refers to a specific dimension d of the velocity vector, rand refers to a random number generated from a uniform distribution in the range $[0, 1]$ and V_{max}^d refers to the clamping limit of dimension d .

3.4. UAPSO – repulsion based particle velocity update

The original PSO algorithm is susceptible to being trapped into a local solution, which could result in slow convergence and might lead to premature convergence, especially for problems with complexity [78]. Some researchers tried to avoid this by introducing a chaotic factor to the velocity-update equation [79] and others by modifying the equation, such as the Repulsive Particle Swarm Optimization (RPSO) [80]. RPSO seeks to enhance global exploration by introducing repulsion between the particles, which could increase the probability of determining the global optima. The velocity update equation in RPSO is modified as follows:

$$v^{(i, k+1)} = w v^{(i, k)} + c_1 \text{rand}_1^{(k)} (x_p^{(i, k)} - x^{(i, k)}) + c_2 \text{rand}_2^{(k)} (x_p^{(j, k)} - x^{(i, k)}) + c_3 \text{rand}_3^{(k)} v_d \quad (19)$$

Step 1	Initialization Randomise $\mathbf{x}^{(i,0)}$ Set $\mathbf{v}^{(i,0)} = 0$ and $\mathbf{x}^{(i,0)} = \mathbf{x}_p^{(i,0)}$ Determine $\mathbf{x}_G^0$
Step 2	Iterative Cycle $k = 1$: iterations FOR each particle i Calculate the Evolutionary State (Eq. (14)) Update the Acceleration Coefficients (Eqs. (16-17)) Update the Particle Velocity (Eq. (20)) Update the Particle Location (Eq. (2)) Calculate objective function $f(\mathbf{x}^{(i,k)})$ If $f(\mathbf{x}^{(i,k)}) < f(\mathbf{x}_p^{(i,k)})$, $\mathbf{x}_p^{(i,k)} = \mathbf{x}^{(i,k)}$ end IF If $f(\mathbf{x}^{(i,k)}) < f(\mathbf{x}_G^k)$, $\mathbf{x}_G^k = \mathbf{x}^{(i,k)}$ end IF If $\mathbf{v}_d^{(i,k+1)} = 0$, Reinitialize the Particle's Velocity (Eq. (18)) end IF end FOR
Step 3	$k = k + 1$ Repeat Step 2 until termination. Termination criteria $k = \text{Max } k$

Fig. 5. Pseudocode of UAPSO.

where $\mathbf{x}_p^{(j,k)}$ is the best solution of a randomly selected particle, c_3 is an acceleration coefficient ($c_3 = 0.5$), rand_3 is a random number between a range of $[0, 1]$ and $\mathbf{v}_d$ is a random component of a particle's velocity. Here $c_2 = -1.43$. The third component in the right-hand side of Eq. (19) creates a repulsion effect between a random particle's best solution and that of a particle. The third term attempts to navigate the particles in such a way as to avoid being trapped in a local optimum, and the fourth term generates noise to enhance exploration across the search area. However, as can be seen in Eq. (19), the equation lacks any global information, and this restricts the overall learning between particles. In UAPSO, a fourth term is introduced into the particle velocity-update equation to both enhance the global exploration and increase robustness. By using the evolutionary state, it is possible to adapt the equation based on the proximity of the latest feasible solution to that of the best solutions. The modified particle velocity-update equation is displayed as follows:

$$\mathbf{v}^{(i,k+1)} = ES^{(i,k)} \mathbf{v}^{(i,k)} + c_1 \text{rand}_1^{(k)} (\mathbf{x}_p^{(i,k)} - \mathbf{x}^{(i,k)}) + c_2 \text{rand}_2^{(k)} (\mathbf{x}_G^k - \mathbf{x}^{(i,k)}) - (1 - ES^{(i,k)}) (\mathbf{x}_G^k - \mathbf{x}_p^{(i,k)}) \quad (20)$$

In Eq. (20), particles are repulsed based on two factors: (1) the difference between the personal and global best solutions, and (2) the value of the evolutionary state. At higher evolutionary states, a lower repulsion is experienced, and at lower evolutionary states a higher repulsion is experienced. Fig. 5 displays the pseudocode of the proposed method.

3.5. Sensitivity analysis

As discussed in Section 2.3, the control parameters need to be tuned correctly for the algorithm to perform effectively and to avoid premature convergence. In order to study the parameters in UAPSO, a sensitivity analysis using the one-at-a-time technique is implemented [81,82]. The sensitivity measure is obtained by increasing a parameter with a given increment while leaving the other parameters fixed and then quantifying the output change in the model. This approach allows the user to know which parameter is responsible for the failure of the model. The swarm size, maximum number of iterations, and velocity-clamping limit are the control parameters in UAPSO. Three cases of each parameter are studied while keeping the others fixed. This analytical analysis is executed by solving the constrained minimization problem, used extensively in literature [83–86]. The problem description is as follows:

$$\begin{aligned} \min f(x) &= (x_1 - 10)^2 + 5(x_2 - 12)^2 + x_3^4 + 3(x_4 - 11)^2 \\ &\quad + 10x_5^6 + 7x_6^2 + x_7^4 - 4x_6x_7 - 10x_6 - 8x_7 \\ c(x)_1 &= 127 - 2x_1^2 - 3x_2^4 - x_3 - 4x_4^2 - 5x_5 \geq 0 \\ c(x)_2 &= 282 - 7x_1 - 3x_3 - 10x_3^2 - x_4 + x_5 \geq 0 \\ c(x)_3 &= 196 - 23x_1 - x_2^2 - 6x_6^2 + 8x_7 \geq 0 \\ c(x)_4 &= -4x_1^2 - x_2^2 + 3x_1x_2 - 2x_3^2 - 5x_6 + 11x_7 \leq 0 \end{aligned} \quad (21)$$

where $-10 \leq x_i \leq 10$ ($i = 1, 2, \dots, 6$). Each case is executed with 30 runs to extract statistical data. Table 2, 3, and 4 contain the analytical experiment results of varying velocity-clamping limit, the maximum number of iterations and swarm size, respectively.

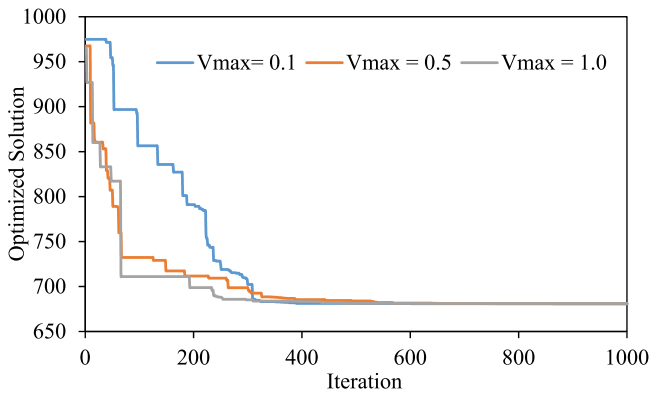


Fig. 6. Effect of velocity-clamping limit with respect to convergence.

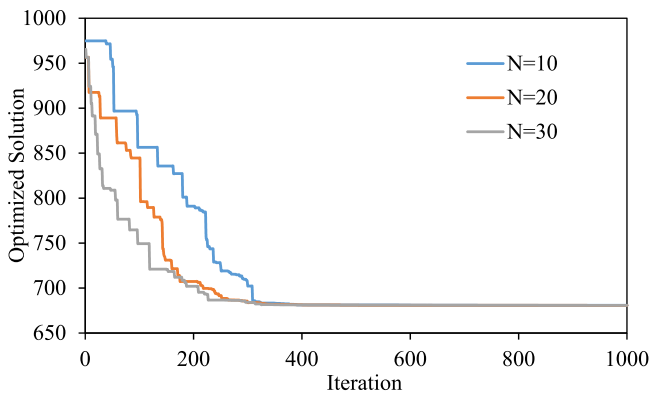


Fig. 7. Effect of swarm size with respect to convergence.

The velocity clamping-limit analysis shows that as the particles freedom to explore the design space increases, the chances of them overflying the global solution is higher, and this is evident from Table 2 and agrees with [60,74]. By increasing the simulation time, the probability of discovering the global solution, or more optimal solution, increases as can be seen in Table 3 and this led to decreasing both the mean function value and standard deviation which translates to a more efficient performance from the algorithm. A larger swarm size increases the diversity, which increases the chances of a particle discovering the optimal solution, as seen in Table 4. The effect of the velocity-clamping limit and swarm size in convergence is displayed in Figs. 6 and 7, where it is seen that a larger $V_{\max}$ and N allows the particles to converge much quicker than when a smaller value is set. The performance of UAPSO with varying control parameters shows that having a larger swarm size might lead the particles to find the optimal global solution instead of increasing the simulation time. However, to form a conclusive hypothesis of the control parameters, more analytical studies are required.

4. Benchmark engineering constrained optimization problems

In this section, the performance of UAPSO is examined by solving multiple constrained optimization problems. Four standard benchmark constrained minimization functions, and four constrained engineering problems used in literature are considered here to compare the performance of UAPSO with other popular algorithms. Descriptions of the considered state-of-the-art algorithms are given in Table 5. These engineering problems include objective functions and constraints with various forms (continuous and mixed-integer problems) and structure (linear and non-linear). The performance of UAPSO will be analyzed by using

statistical means and function evaluations, FEs. The statistical data considered include the best function value, worst function value, mean function value, and standard deviation. Furthermore, the design variable(s) of the best solution, the constraint(s) value of the corresponding best solution and the number of function evaluations are also studied. Function evaluation is the product of the swarm size and the maximum number of iterations. Convergence speed is evaluated based on the algorithm that required the fewest FEs to obtain the best global solution. The algorithm evaluation-mechanism is as follows:

1. The algorithms that managed to achieve the best function value are preferred over ones that did not determine it.
2. The algorithm with the quickest convergence speed is preferred.
3. Secondary to the convergence speed is the robustness and efficiency of the algorithm — this is established by a low standard deviation, worst function value and mean function value.

The following simulations were formulated using the numerical computing software MATLAB (R2017a — Academic Version), and run on Intel Core i7 with 16 GB RAM. The constrained optimization benchmark problems were independently run 100 times to extract the statistical performance of the proposed method. Table 6 contains the user parameters applied to the considered benchmark problems. Two cases of UAPSO are studied: UAPSO₁ for convergence speed and UAPSO₂ for robustness and efficiency. The maximum velocity is set to the dynamic range of the design variables for all four problems. The problem descriptions of the eight selected benchmark problems are given in Appendix A.

4.1. Constrained optimization functions

The performance of UAPSO is compared with HEA-ACT, HuS, HS, PSO-DE, DSS-MDE, and MBA. The numerical results are given in Table 7; where the best, worst, and mean function value, standard deviation, and number of function evaluations are displayed. The optimal solution obtained by UAPSO for each benchmark optimization function are:

- For g01, $\mathbf{x}^* = [0.822882, 0.911436]$ and $f(\mathbf{x}^*) = 1.393454$
- For g02, $\mathbf{x}^* = [-0.707144, 0.500053]$ and $f(\mathbf{x}^*) = 0.750000$
- For g03, $\mathbf{x}^* = [2.246825, 2.381851]$ and $f(\mathbf{x}^*) = 13.590842$
- For g04, $\mathbf{x}^* = [2.328843, 1.952281, -0.484203, 4.363833, 0.623070, 1.032938, 1.590852]$ and $f(\mathbf{x}^*) = 680.630600$

As can be seen in Table 7, UAPSO manages to consistently obtain the global optimum over the 100 independent runs in all four test functions. Furthermore, the statistical performance produced shows the robustness and efficiency of the proposed method. In terms of convergence speed, UAPSO₁ managed to outperform all the other methods in g01, g02, and g03 since it obtained the global optimum in the fewest number of function evaluations. However, PSO-DE managed to have the quickest convergence speed for g04. Comparing UAPSO₁ and MBA shows the superiority of the proposed method as it produced an optimal solution closer to the global optimum with a lower mean and worst function value in fewer FEs. The statistical performance, reported in Table 7, show that for g01 UAPSO₂ produced better mean and worst function value, and standard deviation than HuS, HS, MBA, and UAPSO₁. For g02, UAPSO outperformed PSO-DE, DSS-MDE, MBA, and UAPSO₁ but HEA-ACT produced a lower worst and mean solution. The best solution obtained by PSO-DE and DSS-MDE in g02 is not the global optimum and thus failed in solving the problem. In g03, UAPSO₂ produced better statistical performance than HS, MBA, and UAPSO₁. As for g04,

Table 2 $V_{\max}$ Analysis with $\text{Max}_k = 1,000$ and $N = 10$.

$V_{\max}$	Best	Worst	Mean	Standard deviation
0.1	680.683256	683.606909	681.429537	5.92 E-01
0.5	680.739301	681.918860	681.337727	3.63 E-01
1.0	680.973755	682.387638	681.592901	4.74E-01

Table 3 Max_k analysis with $V_{\max} = 0.1$ and $N = 10$.

Max. no. of iterations	Best	Worst	Mean	Standard deviation
1000	680.683256	683.606909	681.429537	5.92 E-01
2000	680.663350	681.287223	680.905790	1.58 E-01
3000	680.661520	680.870716	680.772714	6.50 E-02

Table 4 N analysis with $\text{Max}_k = 1000$ and $V_{\max} = 0.1$.

Swarm size	Best	Worst	Mean	Standard deviation
10	680.683256	683.606909	681.429537	5.92 E-01
20	680.673723	680.967637	680.845991	8.99 E-02
30	680.654888	680.883207	680.786398	6.85 E-02

HEA-ACT, DSS-MDE, PSO-DE, and UAPSO₂ managed to obtain the global optimum; DSS-MDE produced the lowest standard deviation from them and thus performed the most robust and efficient. UAPSO₂ failed to determine the global optimum but managed to report better statistical performance than UAPSO₁, MBA, and HS. The best solution obtained by HuS is closer to the global optimum than the one produced by UAPSO₂. Fig. 8 depicts the convergence history of the best-obtained solutions by UAPSO₁ and UAPSO₂. UAPSO₂, due to its larger swarm size, consistently converges to the global optimum faster than UAPSO₁.

4.2. Speed-reducer

The speed-reducer optimization problem [98] is a common benchmark example used for testing and comparing optimization methods and is displayed in Fig. 9. In this problem, the objective is to minimize the weight of the speed-reducer while optimizing seven design variables – face width (b), module of teeth (m), number of teeth in the pinion (z), length of the first shaft between bearings (l_1), length of the second shaft between bearings (l_2), and the diameter of first (d_1) and second shafts (d_2), respectively. All design variables are continuous apart from z , which is an integer value. Since this is a mixed integer problem, the UAPSO algorithm is modified to initially treat the discrete variables (x_1 and x_2) the same as the continuous variables during the iterative process, but at the end of an iteration, they are rounded off to the nearest integer value. The truncation approach is commonly used in literature for tackling discrete/integer variables [26,99]. Furthermore, the problem is constrained by 11 constraints. The best solution obtained by UAPSO is $x = [3.500000, 0.700000, 17.000000, 7.300000, 7.715320, 3.350215, 5.286654]$ with $f(x) = 2994.471066$ in both cases of UAPSO. Comparison between the best solutions obtained by UAPSO with the other algorithms is presented in Table 8. The considered algorithms are HEA-ACT, DELC, MDE, PSO-DE, BSAISA, ABC, MBA, DSS-MDE, and WSA. The statistical performance of the considered methods and the proposed method is presented in Table 9. As shown in Table 8, all considered methods satisfied the constraints applied in this problem. BSAISA constraint values are not available; however, the authors of BSAISA claim that BSAISA satisfied all constraints [83]. In terms of obtaining the best solution, DELC, BSAISA and UAPSO managed to obtain the

best solution, $f(x) = 2994.471066$, while HEA-ACT, MDE, PSO-DE, ABC, MBA, DSS-MDE, and WSA could not. UAPSO₁ managed to obtain the global solution with the quickest convergence speed of 15,000 FEs. As for the quality of results, UAPSO₂ performed the best compared to the considered methods by outperforming DELC and DSS-MDE with a lower standard deviation. Fig. 10 depicts the convergence curves of UAPSO₁ and UAPSO₂ for the speed-reducer problem, where the optimized solution refers to the best-obtained solution by the swarm. UAPSO₂ displays quicker convergence towards the global solution ($f(x) = 2994.471066$) due to its larger swarm size.

4.3. Pressure vessel

The pressure vessel design problem consists of a nonlinear objective function with three linear equality equations and one nonlinear inequality equation [100]. In this problem, a cylindrical pressure-vessel is capped at the two ends by hemispherical heads as displayed in Fig. 11. The objective of this problem is to minimize the overall cost of manufacturing the pressure-vessel, and it consists of four design variables. Among the four design variables, the thickness of the shell T_s (x_1) and the thickness of the head T_h (x_2) are discrete variables that are integer multiples of 0.0625. The inner radius R (x_3) and the length of the cylindrical section of the vessel L (x_4) are continuous variables. The integer variable is handled using the truncated method [26]. The best solution obtained by UAPSO is $x = [0.8125, 0.4375, 42.0984, 176.6366]$ with $f(x) = 6059.7143$ in both cases of UAPSO. Comparison between the best solutions obtained by UAPSO with the other algorithms is presented in Table 10. The considered algorithms are the CPSO, CDE, GA-DT, BSA-SA-, BSAISA, G-QPSO, HPSO, PSO-DE, CSA, ABC, and DELC. The statistical performance of the considered methods and the proposed method is presented in Table 11. As shown in Table 10, all considered methods satisfied the constraints applied in this problem. In terms of obtaining the best solution, BSA-SA-, BSAISA, HPSO, PSO-DE, DELC, and UAPSO managed to obtain the best solution, $f(x) = 6059.7143$, while CPSO, CDE, GA-DT, G-QPSO, CSA, and ABC could not. UAPSO₁ managed to obtain the global solution with the quickest convergence speed of 15,000 FEs. As for the quality of results, UAPSO₂ performed better than BSA-SA-, BSAISA, HPSO, and UAPSO₁ since it produced a lower standard deviation and mean function value. However,

Table 5
Selected state-of-the-art algorithms for the comparative study.

Algorithm	Description
Hybrid Evolutionary Algorithm and Adaptive Constraint-Handling Technique (HEA-ACT) [87]	In HEA-ACT, an offspring population is generated by applying a simplex crossover and two mutation operators. Additionally, an adaptive constraint handling technique based on the feasibility of the population is applied.
Differential Evolution with Level Comparison (DELIC) [85]	In DELIC, a constrained optimization problem is solved by applying a level comparison and using the differential evolution method to determine the global solution.
Modified Differential Evolution (MDE) [88]	In MDE, each parent is allowed to produce more than one offspring, and the best performing offspring is selected based on a set of feasibility rules. Apart from this mechanism, MDE utilizes the original mechanism of the differential evolution.
Backtracking Search Algorithm combined with the Self-Adaptive ε constrained method (BSA-SA ε) [89]	The backtracking search algorithm comprises two selection processes where former solutions are used in the mutation operation before the application of the crossover process. Additionally, a self-adapting scheme is employed to control the ε value in the ε -constrained method.
Backtracking Search Optimization Algorithm Inspired by Simulated Annealing (BSAISA) [83]	Here, the simulated annealing method is used to adapt the mutation factor in the backtracking search algorithm. Furthermore, a self-adaptive ε -constrained method is employed to handle constraints.
Co-Evolutionary Differential Evolution (CDE) [90]	Similar to CPSO, here, two populations are generated, one for adapting penalty factors and the other evolving decision solutions.
Genetic Algorithm based on Dominance Tournament-Selection (GA-DT) [91]	By applying a dominance-based strategy, constraints are incorporated into the objective function of the genetic algorithm.
Mine Blast Algorithm (MBA) [84]	MBA simulates the explosion of mine bombs and aims to locate the mine that would create the biggest explosion. Moreover, a set of feasibility rules are utilized for handling constraints.
Hunting Search (HuS) [92]	HuS simulates the hunting/prey scenario that occurs in nature as cooperation between the hunters leads to locating the prey, where hunters are considered as search positions and prey as potential solutions.
Harmony Search (HS) [93]	HS mimics the process a musician undergoes of finding the optimal state of harmony, which represents the global optimum. A re-initialization scheme is implemented whenever a harmony fails to satisfy all constraints.
Differential Evolution with Dynamic Stochastic Selection (DSS-MDE) [94]	In DSS-MDE, a dynamic stochastic selection method was applied to multilevel differential evolution for solving constrained optimization problems.
Artificial Bee Colony (ABC) [86]	ABC imitates the foraging behavior of a colony of bees, which include employed bees, onlooker bees, and scouts, and Deb's approach is applied to handle constraints.
Weighted Superposition Attraction (WSA) [95]	WSA is a swarm-based algorithm that is based on the superposition principle in combination with the attracted movement of agents. It was concluded that WSA is not sensitive to the type of constraint handling technique employed.
Crow Search Algorithm (CSA) [96]	CSA simulates the foraging behavior of crows, and a set of feasibility rules are employed for the handling of constraints.
Thermal Exchange Optimization (TEO) [97]	TEO inspiration comes from Newton's law of cooling by considering agents as cooling objects and environments, and thermal exchange occurs between them. The new temperature represents the updated position of the agent. The penalty approach is employed for constraint handling.

PSO-DE and DELC produced a lower worst function value and standard deviation. Overall, in terms of robustness and effectiveness, UAPSO₂ outperformed all the considered methods apart from PSO-DE and DELC. Fig. 12 depicts the convergence curves of UAPSO₁ and UAPSO₂ for the speed-reducer problem, where the optimized solution refers to the best-obtained solution by the swarm. UAPSO₂ displays quicker convergence towards the global solution ($f(x) = 6059.7143$) due to its larger swarm size.

4.4. Tension/compression spring

The tension/compression spring design problem consists of a nonlinear objective function with four nonlinear inequality equations [101] and is illustrated in Fig. 13. The objective of this problem is to minimize the structural weight of the coil spring, and it consists of three continuous design variables. The design variables

are the wire diameter d (x_1), mean coil diameter D (x_2), and the number of active coils N (x_3). Furthermore, the spring is subjected to constraints on minimum deflection, shear stress, surge frequency, and the outer diameter of the spring. The best solution obtained by UAPSO is $x = [0.051712, 0.357264, 11.257028]$ with $f(x) = 0.012665$ in both cases of UAPSO. Comparison between the best solutions obtained by UAPSO with the other algorithms is presented in Table 12. The considered algorithms are the CPSO, MDE, PSO-DE, CDE, BSAISA, BSA-SA ε , GA-DT, HPSO, WSA, DSS-MDE, HEA-ACT, CSA, TEO, ABC, G-QPSO, and DELC. The statistical performance of the considered methods and the proposed method is presented in Table 13. As shown in Table 12, all considered methods satisfied the constraints applied in this problem. In terms of obtaining the best solution, MDE, PSO-DE, BSAISA, BSA-SA ε , HPSO, WSA, DSS-MDE, HEA-ACT, CSO, TEO, ABC, DELC, and UAPSO managed to obtain the best solution, $f(x) = 0.012665$, while CPSO, CDE, GA-DT, and G-QPSO could

Table 6

User parameters for the benchmark design problems.

Design problem		Swarm size	Max. no of iterations	FEs
g01	UAPSO ₁	4	500	2,000
	UAPSO ₂	30	500	15,000
g02	UAPSO ₁	8	500	4,000
	UAPSO ₂	100	500	50,000
g03	UAPSO ₁	8	500	4,000
	UAPSO ₂	30	500	15,000
g04	UAPSO ₁	25	2,000	50,000
	UAPSO ₂	70	2,000	140,000
Speed Reducer	UAPSO ₁	5	3,000	15,000
	UAPSO ₂	10	3,000	30,000
Pressure Vessel	UAPSO ₁	5	3,000	15,000
	UAPSO ₂	10	3,000	30,000
Tension/Compression Spring	UAPSO ₁	9	1,000	9,000
	UAPSO ₂	24	1,000	24,000
Welded Beam	UAPSO ₁	15	1,000	15,000
	UAPSO ₂	45	1,000	45,000

Table 7

Experimental simulation results and comparison with six state-of-the-art algorithms.

Method	Performance	g01	g02	g03	g04
HEA-ACT	Best	N/A	0.750	N/A	680.630
	Worst	N/A	0.750	N/A	680.630
	Mean	N/A	0.750	N/A	680.630
	Standard deviation	N/A	3.40 E−16	N/A	5.80 E−13
	FEs	N/A	50,000	N/A	200,000
HuS	Best	1.3935	N/A	N/A	680.6300771
	Worst	N/A	N/A	N/A	N/A
	Mean	N/A	N/A	N/A	N/A
	Standard deviation	N/A	N/A	N/A	N/A
	FEs	3,750	N/A	N/A	22,000
HS	Best	1.3770	N/A	13.590845	680.6413574
	Worst	N/A	N/A	N/A	N/A
	Mean	N/A	N/A	N/A	N/A
	Standard deviation	N/A	N/A	N/A	N/A
	FEs	40,000	N/A	15,000	160,000
PSO-DE	Best	N/A	0.749999	N/A	680.6300574
	Worst	N/A	0.750001	N/A	680.6300574
	Mean	N/A	0.749999	N/A	680.6300574
	Standard deviation	N/A	2.50 E−07	N/A	4.60 E−13
	FEs	N/A	70,100	N/A	140,100
DSS-MDE	Best	N/A	0.749999	N/A	680.630
	Worst	N/A	0.749999	N/A	680.630
	Mean	N/A	0.749999	N/A	680.630
	Standard deviation	N/A	0	N/A	2.50 E−13
	FEs	N/A	225,000	N/A	350,000
MBA	Best	1.393465	0.750000	13.590842	680.6322
	Worst	1.393465	0.750011	13.592670	680.7882
	Mean	1.393465	0.750003	13.591027	680.6620
	Standard deviation	0	3.29 E−06	3.64 E−04	3.30E−02
	FEs	2,140	6,405	4,560	71,750
UAPSO ₁	Best	1.393449	0.750000	13.590842	680.6310805
	Worst	1.413494	0.750679	13.594705	680.6738750
	Mean	1.399958	0.750151	13.591818	680.6517405
	Standard deviation	5.59 E−03	1.46 E−04	1.36 E−03	1.18 E−02
	FEs	2,000	4,000	4,000	50,000
UAPSO ₂	Best	1.393449	0.750000	13.590842	680.630600
	Worst	1.393449	0.750006	13.590865	680.643377
	Mean	1.393449	0.750002	13.590849	680.636665
	Standard deviation	7.73 E−14	1.84 E−06	6.85 E−06	3.85 E−03
	FEs	15,000	50,000	15,000	350,000

not. UAPSO₁ managed to obtain the global solution with the quickest convergence speed of 9000 FEs. As for the quality of results, UAPSO₂ performed better than UAPSO₁, MDE, HPSO, WSA, DSS-MDE, CSA, TEO, and ABC. However, PSO-DE, HEA-ACT, and DELC produced a lower worst and mean function value and standard deviation. Overall, in terms of robustness and effectiveness, UAPSO₂ outperformed the majority of the considered methods.

Fig. 14 depicts the convergence curves of UAPSO₁ and UAPSO₂ for the speed-reducer problem, where the optimized solution refers to the best-obtained solution by the swarm. UAPSO₂ displays quicker convergence towards the global solution ($f(x) = 0.012665$) due to its larger swarm size.

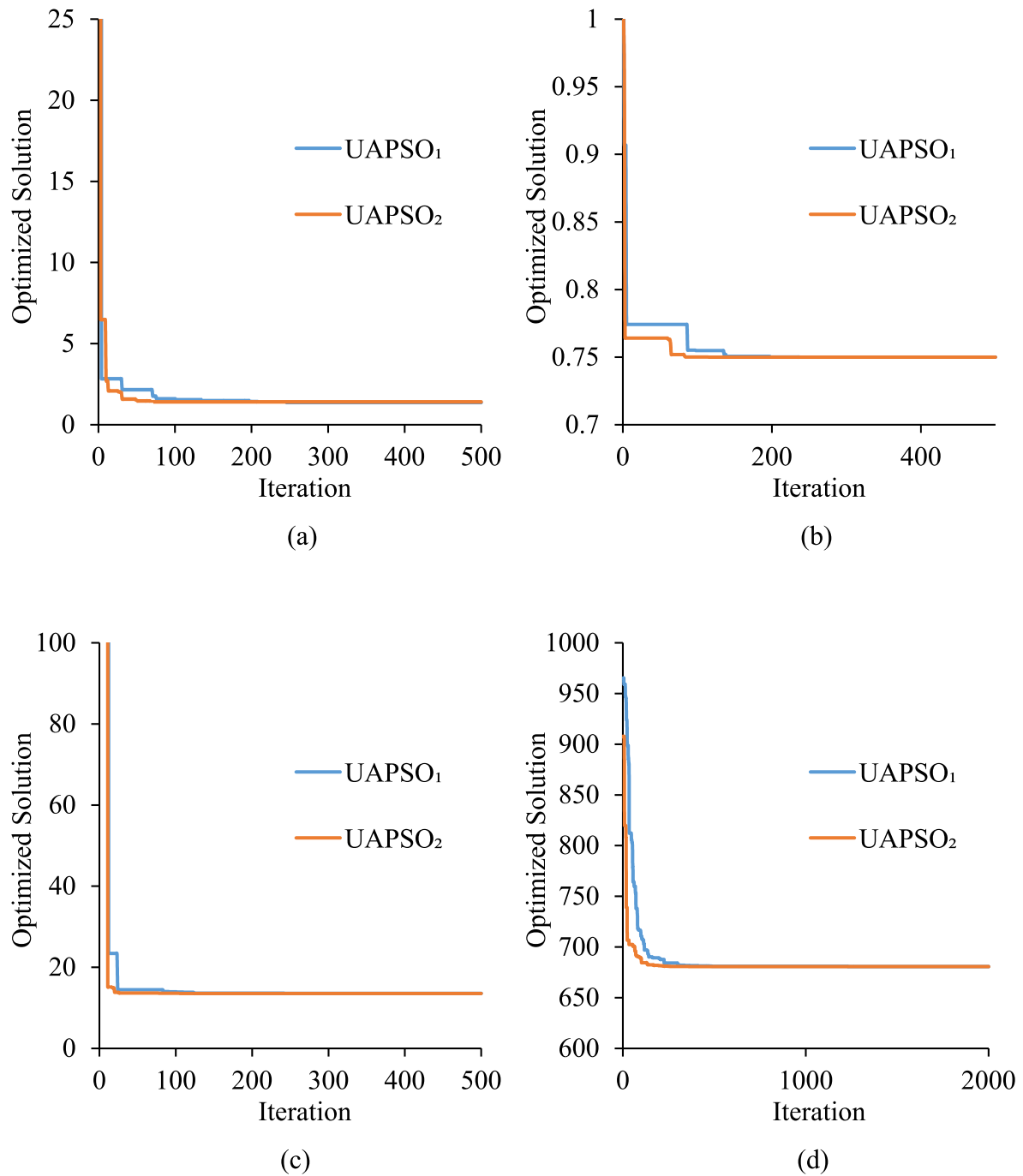


Fig. 8. The convergence plots of the best obtained solution by UAPSO₁ and UAPSO₂ on (a) g01, (b) g02, (c) g03, and (d) g04.

4.5. Welded beam

The welded beam design problem consists of a nonlinear objective function with seven nonlinear inequality equations [102] and is displayed in Fig. 15. The objective of this problem is to minimize the cost subjected to constraints applied on shear stress (τ), bending stress in the beam (σ), buckling load on the bar (P_c), end deflection of the beam (δ) and side constraints. The design variables included in this problem are the height of the weld h (x_1), the length of the beam l (x_2), the thickness of the beam t (x_3) and the breadth of the beam b (x_4). The best solution obtained by UAPSO is $x = [0.205730, 3.470489, 9.036624, 0.205730]$ with $f(x) = 1.724852$ in both cases of UAPSO. Comparison between the best solution obtained by UAPSO with the other considered

algorithms is presented in Table 14. The considered algorithms are the CPSO, HPSO, PSO-DE, MBA, BSAISA, BSA-SA⁺, CDE, GA-DT, WSA, DELC, CSA, TEO, and ABC. The statistical performance of the considered methods with respect to the proposed method is presented in Table 15. As shown in Table 14, all considered methods satisfied the constraints applied in this problem. In terms of obtaining the best solution, HPSO, PSO-DE, BSAISA, BSA-SA⁺, WSA, DELC, CSA, and ABC managed to obtain the best solution, $f(x) = 1.724852$, while CPSO, MBA, CDE, GA-DT, and TEO could not. UAPSO₁ managed to obtain the global solution with the quickest convergence speed of 15,000 FEs. As for the quality of results, UAPSO₂ performed better than UAPSO₁, HPSO BSAISA, WSA, and ABC since it produced a stronger statistical performance. However, PSO-DE, BSA-SA⁺, DELC, and CSA produced a lower worst

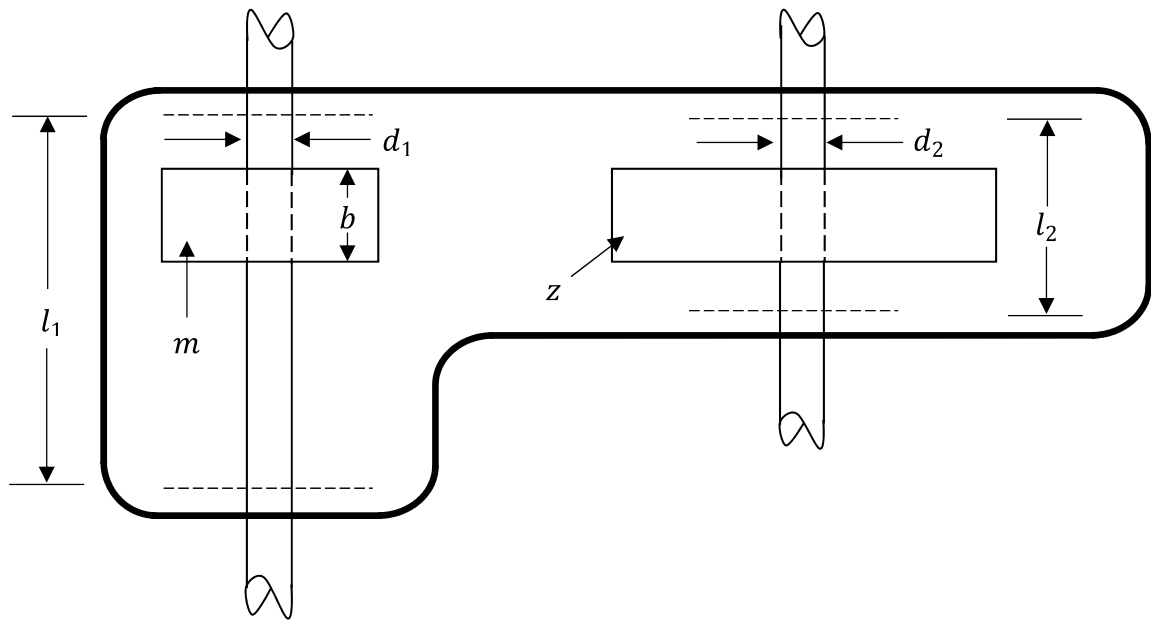


Fig. 9. Schematic representation of the speed-reducer problem.

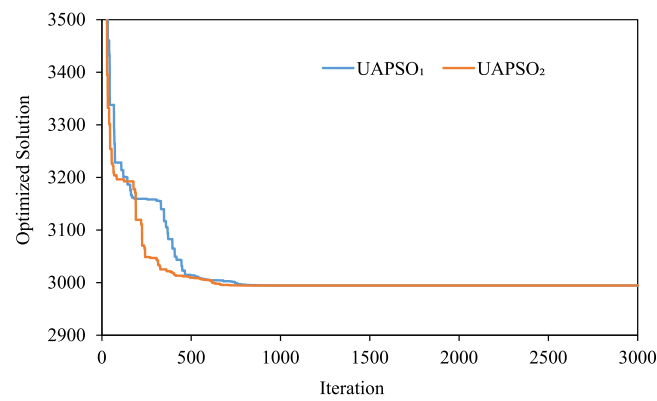


Fig. 10. Convergence curves of UAPSO₁ and UAPSO₂ for the speed-reducer problem.

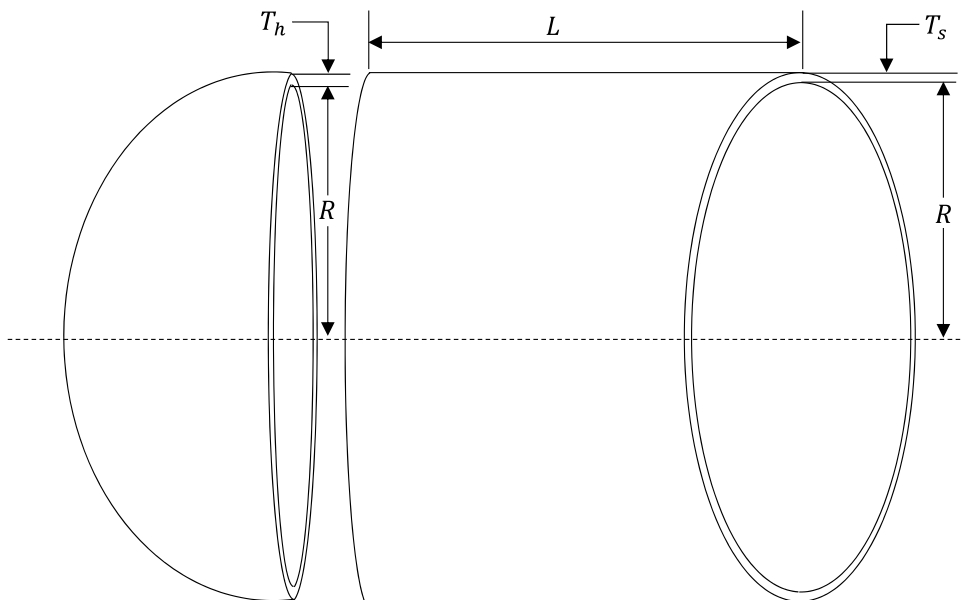


Fig. 11. Schematic representation of the pressure vessel problem.

Table 8

Comparison of the best solutions for the speed-reducer problem obtained by UAPSO with five state-of-the-art algorithms.

D. V.	HEA-ACT	DELC	MDE	PSO-DE	BSAISA	UAPSO
x_1	3.500023	3.500000	3.500010	3.500000	3.500000	3.500000
x_2	0.700000	0.700000	0.700000	0.700000	0.700000	0.700000
x_3	17.000013	17.000000	17.000000	17.000000	17.000000	17.000000
x_4	7.300428	7.300000	7.300156	7.300000	7.300000	7.300000
x_5	7.715377	7.715320	7.800027	7.800000	7.715320	7.715320
x_6	3.350231	3.350215	3.350221	3.350215	3.350215	3.350215
x_7	5.286664	5.286654	5.286685	5.286683	5.286654	5.286654
c_1	-0.073923	-0.073915	-0.073918	-0.073915	N/A	-0.073918
c_2	-0.198006	-0.197998	-0.198001	-0.197998	N/A	-0.198001
c_3	-0.499095	-0.499172	-0.499144	-0.499172	N/A	-0.499144
c_4	-0.904643	-0.904644	-0.901471	-0.901472	N/A	-0.901471
c_5	-1.40 E-05	-3.18 E-12	-5.00 E-06	-4.73 E-13	N/A	-5.00 E-06
c_6	-5.00 E-06	-1.12 E-11	-1.00 E-06	-4.68 E-14	N/A	-1.00 E-06
c_7	-0.702500	-0.702500	-0.702500	-0.702500	N/A	-0.702500
c_8	-6.00 E-06	0	-3.00 E-06	0	N/A	-3.00 E-06
c_9	-0.583331	-0.583333	-0.583332	-0.583333	N/A	-0.583332
c_{10}	-0.051378	-0.051326	-0.051345	-0.051326	N/A	-0.051345
c_{11}	-6.00 E-06	0	-0.010856	-0.010852	N/A	-0.010856
$f(x)$	2994.499107	2994.471066	2996.356689	2996.348167	2994.471066	2994.471066

Table 9

Comparison of the statistical performance for the speed-reducer problem produced by UAPSO with nine state-of-the-art algorithms.

Method	Best	Worst	Mean	Standard deviation	FES
HEA-ACT	2994.499107	2994.752311	2994.613368	7.00 E-02	40,000
DELC	2994.471066	2994.471066	2994.471066	1.90 E-12	30,000
MDE	2996.356689	2996.390137	2996.367220	8.20 E-03	24,000
PSO-DE	2996.348167	2996.348204	2996.348174	6.40 E-06	54,350
BSAISA	2994.471066	2994.471067	2994.471095	5.40 E-06	15,860
ABC	2997.058412	N/A	2997.058412	0	30,000
MBA	2994.482453	2999.652444	2996.769019	1.56	6,300
DSS-MDE	2994.471066	2994.471066	2994.471066	3.58 E-12	30,000
WSA	2996.348222	2996.348244	2996.348233	9.11 E-06	500,000
UAPSO ₁	2994.471066	2994.471503	2994.471098	8.38 E-05	15,000
UAPSO ₂	2994.471066	2994.471066	2994.471066	1.83 E-12	30,000

Table 10

Comparison of the Best Solution for the Pressure Vessel Problem obtained by UAPSO with five state-of-the-art algorithms.

D. V.	CPSO	CDE	GA-DT	BSA-SA _e	BSAISA	UAPSO
x_1	0.8125	0.8125	0.8125	0.8125	0.8125	0.8125
x_2	0.4375	0.4375	0.4375	0.4375	0.4375	0.4375
x_3	42.0913	42.0983	42.0974	42.0984	42.0984	42.0984
x_4	176.7465	176.6377	176.6540	176.6366	176.6366	176.6367
c_1	-1.37 E-06	-6.67 E-07	-2.01 E-03	-9.50 E-10	0	-1.14 E-13
c_2	-3.59 E-04	-3.58 E-02	-3.58 E-02	-3.59 E-02	-3.59 E-02	-3.59 E-02
c_3	-118.7687	-3.6830	-24.8861	-1.20 E-04	0	-1.21 E-08
c_4	-63.2535	-63.3623	-63.3459	-63.3634	-63.3634	-63.3634
$f(x)$	6061.0777	6059.7340	6059.9463	6059.7143	6059.7143	6059.7143

Table 11

Comparison of the Statistical Performance for the Pressure Vessel Problem produced by UAPSO with eleven state-of-the-art algorithms.

Method	Best	Worst	Mean	Standard deviation	FES
CPSO	6061.0777	6363.8041	6147.1332	86.45	30,000
CDE	6059.7340	6371.0455	6085.2303	43.01	204,800
GA-DT	6059.9463	6469.3220	6177.2533	130.93	80,000
BSA-SA _e	6059.7143	6116.7804	6074.3682	17.01	80,000
BSAISA	6059.7143	7198.0054	6418.1935	304.00	16,320
G-QPSO	6059.7208	7544.4925	6440.3786	448.47	8,000
HPSO	6059.7143	6288.6770	6099.9323	86.20	81,000
PSO-DE	6059.7143	6059.7143	6059.7143	1.00 E-12	42,100
CSA	6059.7144	7332.8416	6342.4991	384.94	250,000
ABC	6059.7147	N/A	6245.3081	205.00	30,000
DELC	6059.7143	6059.7143	6059.7143	2.10 E-11	30,000
UAPSO ₁	6059.7143	6093.2435	6062.7048	8.61	15,000
UAPSO ₂	6059.7143	6059.7154	6059.7143	1.10 E-04	30,000

and mean function value and standard deviation. Overall, in terms of robustness and effectiveness, UAPSO₂ outperformed the majority of the considered methods. Fig. 16 depicts the convergence curves of UAPSO₁ and UAPSO₂ for the speed-reducer problem,

where the optimized solution refers to the best-obtained solution by the swarm. UAPSO₂ displays quicker convergence towards the global solution ($f(x) = 1.724852$) due to its larger swarm size.

Table 12

Comparison of the Best Solutions for the Tension/Compression Spring Problem obtained by UAPSO with five state-of-the-art algorithms.

D. V.	CPSO	MDE	PSO-DE	CDE	BSAISA	UAPSO
x_1	0.051728	0.051688	0.051689	0.051609	0.051687	0.051712
x_2	0.357644	0.356692	0.356712	0.354714	0.356669	0.357264
x_3	11.244543	11.290483	11.289320	11.410831	11.291824	11.257028
c_1	−8.45 E−04	−0.000000	−1.99 E−09	−3.90 E−05	−6.38 E−10	−7.95 E−08
c_2	−1.26 E−05	−0.000000	−3.93 E−09	−1.83 E−04	−1.53 E−09	−6.06 E−08
c_3	−4.051300	−4.053734	−4.053774	−4.048627	−4.053689	−4.054862
c_4	−0.727090	−0.727747	−0.727733	−0.729118	−0.727763	−0.727350
$f(x)$	0.012675	0.012665	0.012665	0.012670	0.012665	0.012665

Table 13

Comparison of the Statistical Performance for the Tension/Compression Spring Problem produced by UAPSO with sixteen state-of-the-art algorithms.

Method	Best	Worst	Mean	Standard deviation	FES
CPSO	0.012675	0.012924	0.012730	5.20 E−04	240,000
MDE	0.012665	0.012674	0.012666	2.00 E−06	24,000
PSO-DE	0.012665	0.012665	0.012665	1.20 E−08	24,950
CDE	0.012670	0.012790	0.012703	2.70 E−05	240,000
BSAISA	0.012665	0.012668	0.012666	4.90 E−07	9,440
BSA-SA _e	0.012665	0.012666	0.012665	1.62 E−07	80,000
GA-DT	0.012681	0.012973	0.012742	5.90 E−05	8,000
HPSO	0.012665	0.012719	0.012707	1.58 E−05	81,000
WSA	0.012665	0.012678	0.012671	4.06 E−06	500,000
DSS-MDE	0.012665	0.012738	0.012669	1.25 E−05	24,000
HEA-ACT	0.012665	0.012665	0.012665	1.40 E−09	24,000
CSA	0.012665	0.012666	0.012670	1.36 E−06	50,000
TEO	0.012665	0.012715	0.012685	4.41 E−06	15,000
ABC	0.012665	N/A	0.012709	1.28 E−02	30,000
G-QPSO	0.012666	0.015869	0.012996	6.28 E−04	2,000
DELC	0.012665	0.012666	0.012665	1.30 E−07	20,000
UAPSO ₁	0.012665	0.012671	0.012669	1.74 E−06	9,000
UAPSO ₂	0.012665	0.012667	0.012666	6.75 E−07	24,000

Table 14

Comparison of the best solutions for the welded beam problem obtained by UAPSO with five state-of-the-art algorithms.

D. V.	CPSO	HPSO	PSO-DE	MBA	BSAISA	UAPSO
x_1	0.202369	0.20573	0.205730	0.205729	0.205730	0.205730
x_2	3.544214	3.470489	3.470489	3.470493	3.470489	3.470489
x_3	9.04821	9.036624	9.036624	9.036626	9.036624	9.036624
x_4	0.205723	0.20573	0.205730	0.205729	0.205730	0.205730
c_1	−12.839796	−0.025399	−1.51 E−05	−0.001614	0	−5.64 E−05
c_2	−1.247467	−0.053122	−2.88 E−05	−0.016911	0	−7.91 E−05
c_3	−1.49 E−03	0	0	−2.40 E−07	−5.55 E−12	−8.52 E−12
c_4	−3.429347	−3.432981	−3.432984	−3.432982	−3.432984	−3.432983
c_5	−0.079381	−0.08073	−0.080730	−0.080729	−0.080730	−0.080730
c_6	−0.235536	−0.235540	−0.235540	−0.235540	−0.235540	−0.235540
c_7	−11.681355	−0.031555	−1.86 E−05	−0.001464	−5.46 E−12	−1.40 E−05
$f(x)$	1.728024	1.724852	1.724852	1.724853	1.724852	1.724852

Table 15

Comparison of the statistical performance for the welded beam problem produced by UAPSO with thirteen state-of-the-art algorithms.

Method	Best	Worst	Mean	Standard deviation	FES
CPSO	1.728024	1.782143	1.748831	1.29 E−02	240,000
HPSO	1.724852	1.814295	1.749040	4.00 E−02	81,000
PSO-DE	1.724852	1.724852	1.724852	6.70 E−16	66,600
MBA	1.724853	1.724853	1.724853	6.94 E−19	47,340
BSAISA	1.724852	1.724854	1.724852	2.35 E−07	29,000
BSA-SA _e	1.724852	1.724852	1.724852	8.11 E−10	80,000
CDE	1.733461	1.768158	1.824105	2.22 E−02	240,000
GA-DT	1.728226	1.993408	1.792654	7.47 E−02	80,000
WSA	1.724852	3.823218	2.125751	6.53 E−01	500,000
DELC	1.724852	1.724852	1.724852	4.10 E−13	20,000
CSA	1.724852	1.724852	1.724852	1.19 E−15	100,000
TEO	1.725284	1.931161	1.768040	5.82 E−02	15,000
ABC	1.724852	N/A	1.741913	3.10 E−02	30,000
UAPSO ₁	1.724852	1.725456	1.725141	1.67 E−04	15,000
UAPSO ₂	1.724852	1.724853	1.724853	1.62 E−07	45,000

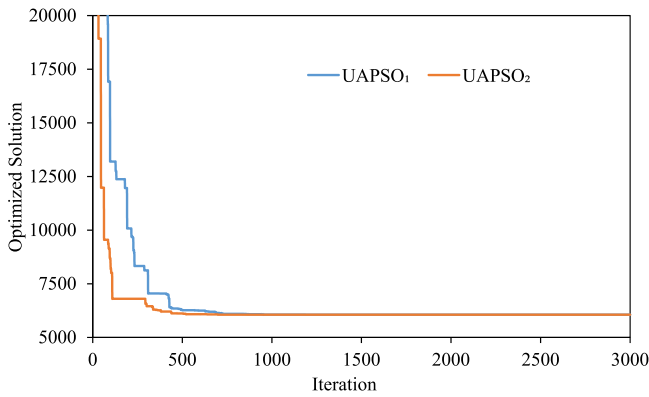


Fig. 12. Convergence curves of UAPSO₁ and UAPSO₂ for the pressure vessel problem.

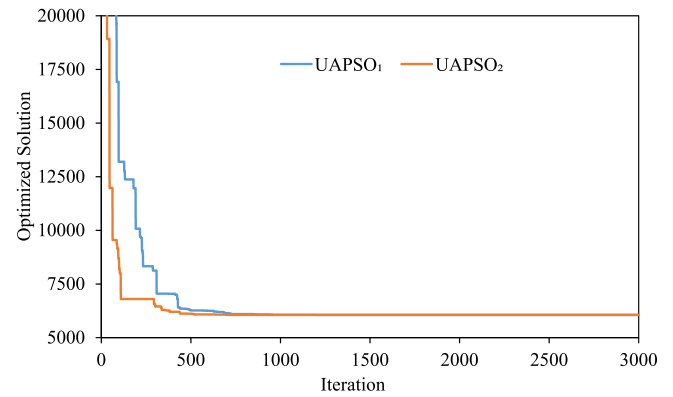


Fig. 14. Convergence curves of UAPSO₁ and UAPSO₂ for the tension/compression spring problem.

5. Application of the proposed method in solving a practical structural optimization problem

The application of the proposed method in solving a real-world structural optimization problem is studied here. The real-world problem involves designing a three-bar truss structure. In theory, any structure that is cheap to produce and reliable is desirable, which is why weight minimization of structural components is highly sought-after. The three-bar truss design problem is statically indeterminate and thus cannot be solved for by merely using the equilibrium equations. Different forms of the three-bar truss problem have been studied previously in literature [101,103,104]. The performance of UAPSO is compared with five other variants of the PSO method, including the original PSO version and the Sequential Quadratic Programming (SQP) method, a derivative based search method.

The objective of this optimization problem is the minimization of the structural weight of a three-bar structure, shown in Fig. 17. The design variables are the cross-sectional areas a_1, a_2, a_3 , and the coordinates x_1, x_2, x_3 of nodes A, B, C. The members are made from the same material with an allowable strength S_a , Young modulus E and density ρ . The structure is subjected to load P at angle θ , which results in horizontal displacement u and vertical displacement v . The system parameters are presented in Table 16.

Due to the statically indeterminate state of the problem, the analytical technique of finite element method was applied to derive the mathematical expressions of the optimization problem presented above. The displacements u and v are implicitly calculated by the following equation:

$$E \sum_{i=1}^3 \frac{a_i}{(x_i^2 + 0.5^2)^{3/2}} \begin{bmatrix} x_i^2 & 0.5x_i \\ 0.5x_i & 0.5^2 \end{bmatrix} \begin{Bmatrix} u \\ v \end{Bmatrix} = \begin{Bmatrix} P \cos \theta \\ -P \sin \theta \end{Bmatrix} \quad (22)$$

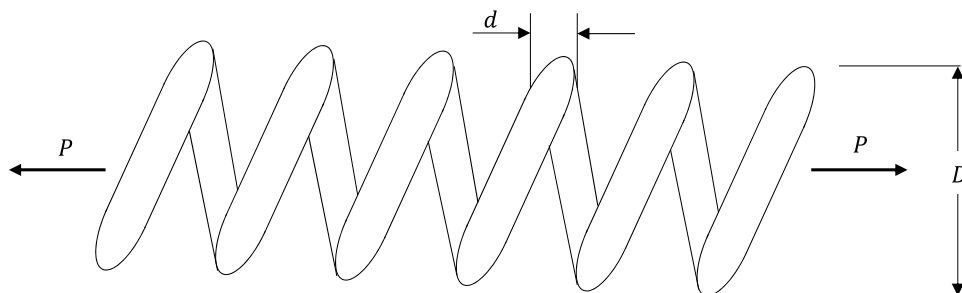


Fig. 13. Schematic representation of the tension/compression spring problem.

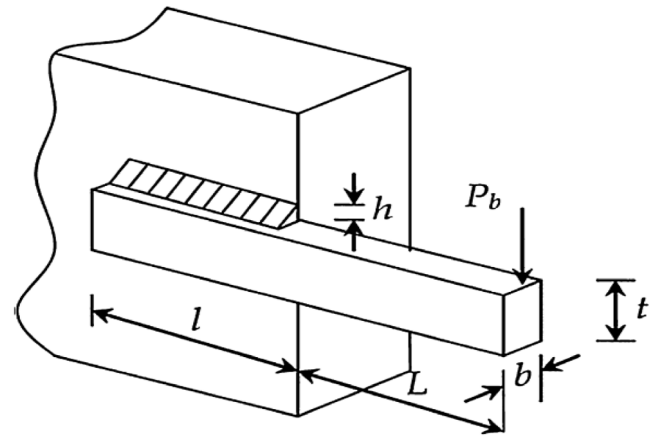


Fig. 15. Schematic representation of the welded-beam problem [102].

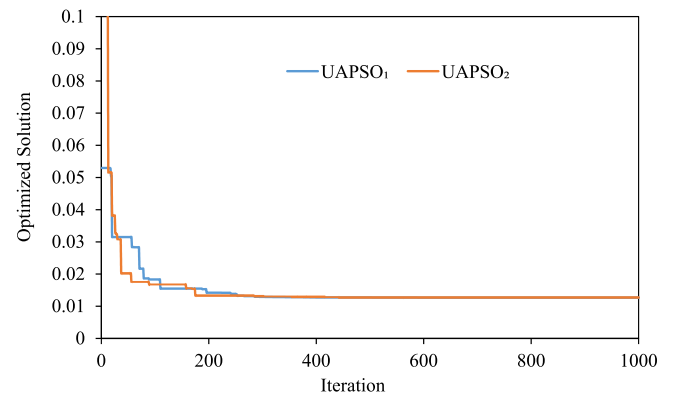


Fig. 16. Convergence curves of UAPSO₁ and UAPSO₂ for the welded beam problem.

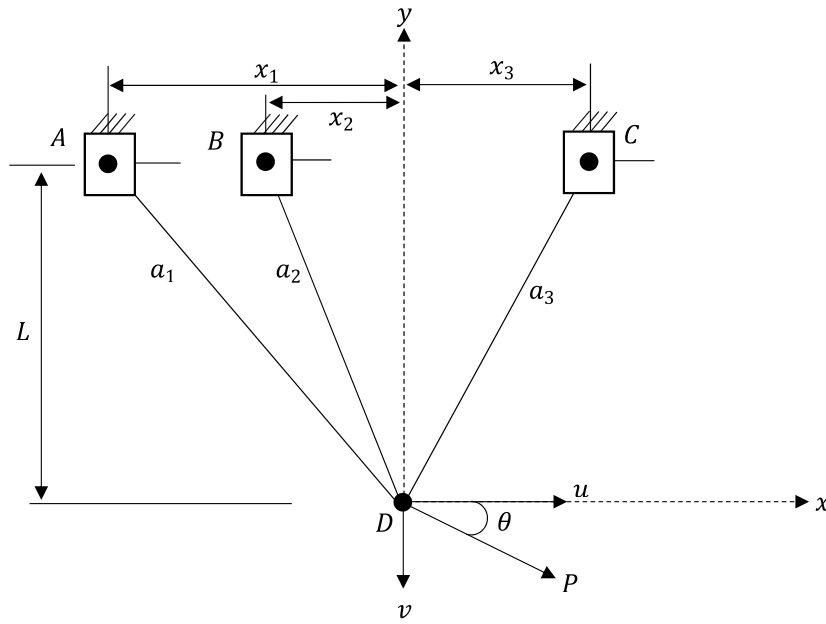


Fig. 17. Schematic diagram of the three-bar truss design problem.

Table 16

System parameters of the three-bar truss design problem.

System Parameter	Parameter Value
Length (L)	0.5 m
Load (P)	70 kN
Angle of Applied Load (θ)	30°
Density (ρ)	7800 kg/m ³
Gravity (g)	9.81 m/s ²
Allowable Strength (S_a)	250 MPa
Young's Modulus (E)	200 GPa
Maximum Relative Displacement (δ)	10 mm
Natural Frequency Threshold (ω_0)	100 Hz

The above equation has been derived from the general equation of a 2D truss element given by the following equation [105]:

$$\frac{EA_i}{L_i} \begin{bmatrix} \cos^2 \theta_i & \cos \theta_i \sin \theta_i & -\cos^2 \theta_i & -\cos \theta_i \sin \theta_i \\ \cos \theta_i \sin \theta_i & \sin^2 \theta_i & -\cos \theta_i \sin \theta_i & -\sin^2 \theta_i \\ -\cos^2 \theta_i & -\cos \theta_i \sin \theta_i & \cos^2 \theta_i & \cos \theta_i \sin \theta_i \\ -\cos \theta_i \sin \theta_i & -\sin^2 \theta_i & \cos \theta_i \sin \theta_i & \sin^2 \theta_i \end{bmatrix} \times \begin{bmatrix} u_i \\ v_i \\ u_j \\ v_j \end{bmatrix} = \begin{bmatrix} P_{ix} \\ P_{iy} \\ P_{jx} \\ P_{jy} \end{bmatrix} \quad (23)$$

where u refers to the horizontal displacement of elements i and j , v refers to the vertical displacement of elements i and j and P refers to the applied load along the directions x and y on elements i and j . The phenomenon of resonance is undesirable since it could lead to system failure [106]. Therefore, the three-bar truss is designed in such a way that its lowest excited natural frequency exceeds a certain threshold. The system has two degrees-of-freedom. Hence, there are two natural frequencies. After deriving the mass and stiffness matrices, the natural frequency of the system can be calculated by solving the following characteristic equation:

$$\mathbf{K} - \mathbf{M}(\omega^2) = 0 \quad (24)$$

where $\mathbf{K}$ is the stiffness matrix, $\mathbf{M}$ the mass matrix, and ω is the frequency vector. By solving the above, the two frequencies are derived, and the lower frequency is subjected to a constraint.

The problem description of the three-bar truss design problem is given in Appendix B.

5.1. Parameter settings

The proposed method is applied to the three-bar truss design problem along with the original PSO [10], PSO-LDIW [19], PSO-NLIW [63], PSO-RIW [61], PSO-CIW [48] and the SQP method [107]. The SQP method is a widely used derivative-based method that utilizes second order information to find the optimal solution. Since SQP is a deterministic method, i.e. the algorithm solves the problem in a deterministic manner, statistical data cannot be generated, and therefore, the optimal solution will only be used in the comparison. SQP was initiated, through the MATLAB function *fmincon*, with a starting design of $[-1, -1, 1, 3.2e - 06, 3.2e - 06, 3.2e - 06]$. The maximum velocity was set to the dynamic range of the design variables. The user parameters include the dynamic range of the inertia weight, swarm size, and the maximum number of iterations for each PSO variant are listed in Table 17. Each method was executed with 100 runs.

5.2. Numerical results

The best solution obtained by UAPSO is $x = [-0.971594, -0.867017, -0.710397, 3.20E-06, 2.74E-04, 3.20E-06]$ with $f(x) = 21.440613$. Comparison between the best solutions obtained by UAPSO, the other considered PSO variants and SQP is presented in Table 18. The statistical performance of the considered PSO variants with respect to the proposed method is presented in Fig. 18. The error bar represents the distribution of results as per the standard deviation. In terms of obtaining the best solution, UAPSO managed to obtain the best solution $f(x) = 21.440613$ among the considered techniques. As for the quality of results, UAPSO performed the best since its statistical performance was superior compared to the rest; UAPSO's mean function value, worst function value, and standard deviation are all lower than the other techniques. Since PSO-LDIW, PSO-NLIW, PSO-RIW, and PSO-CIW introduce more diversity due to their influence on the inertia weight they outperformed the original PSO. Furthermore, the best solution obtained by SQP is lower than

Table 17

Inertia weight range, swarm size and maximum no. of iterations of the PSO variants.

PSO variant	Inertia weight range	Swarm size	Max. no of iterations
Original PSO	1.0	10	2,000
PSO-LDIW	0.4–0.9	10	2,000
PSO-NLIW	0.4–0.9	10	2,000
PSO-RIW	0.5–1.0	10	2,000
PSO-CIW	0.0–1.0	10	2,000
UAPSO	0.0–1.0	10	2,000

Table 18

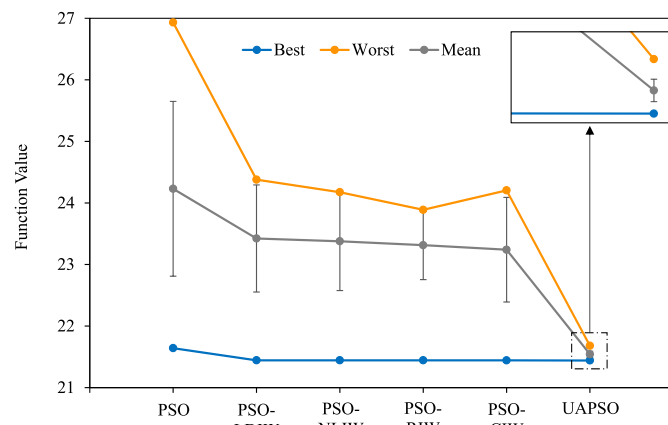
Comparison of the Best Solutions for the three-bar truss design problem obtained by UAPSO, SQP, and the considered PSO variants.

D. V.	SQP	PSO	PSO-LDIW	PSO-NLIW	PSO-RIW	PSO-CIW	UAPSO
x_1	−9.35 E−01	−1.00 E+00	−1.00E+00	−1.00 E+00	−1.00 E+00	−1.00 E+00	−9.72 E−01
x_2	−8.35 E−01	−8.70 E−01	−8.67E−01	−8.67 E−01	−8.67 E−01	−8.67 E−01	−8.67 E−01
x_3	−2.99 E−01	−4.43 E−01	−7.18E−01	−7.18 E−01	−7.18 E−01	−7.41 E−01	−7.10 E−01
x_4	9.32 E−05	3.20 E−06	3.20 E−06	3.20 E−06	3.20 E−06	3.20 E−06	3.20 E−06
x_5	1.87 E−04	2.76 E−04	2.74 E−04	2.72 E−04	2.74 E−04	2.74 E−04	2.74 E−04
x_6	2.02 E−06	3.20 E−06	3.22 E−06	4.87 E−06	3.20 E−06	3.20 E−06	3.20 E−06
$f(x)$	21.5615	21.6408	21.4432	21.4432	21.4432	21.4429	21.4406

Table 19

Sum of violated constraints by the considered PSO Variants and SQP.

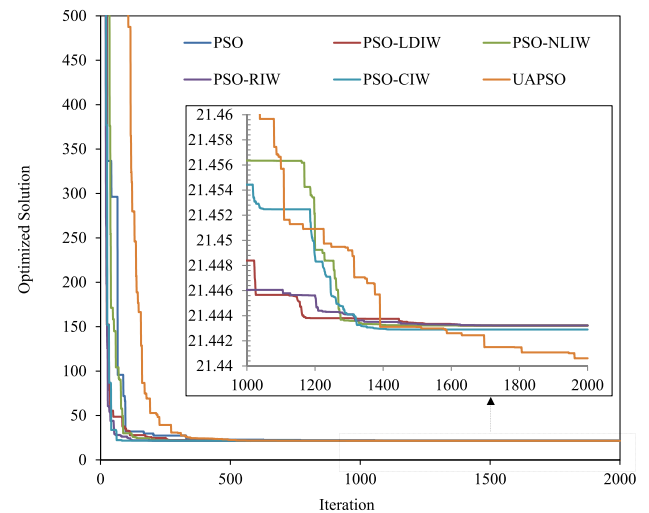
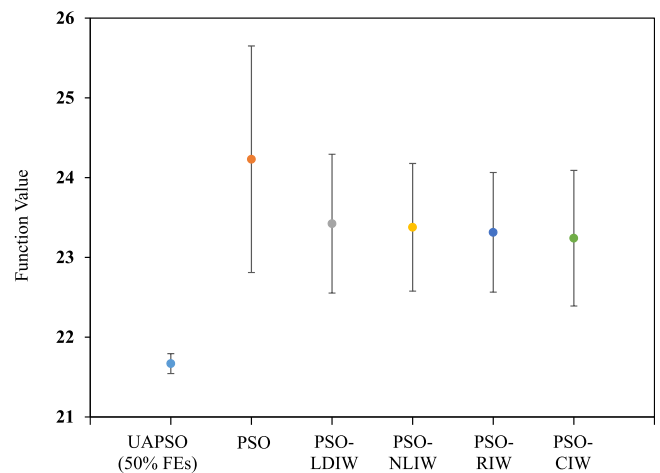
Method	Sum of violated constraints
SQP	2.24 E+04
PSO	0
PSO-LDIW	6.42 E−05
PSO-NLIW	6.91 E−05
PSO-RIW	0
PSO-CIW	0
UAPSO	0

**Fig. 18.** Statistical performance of UAPSO and the considered PSO variants in the three-bar truss problem.

the PSO variants, which is due to their stochastic nature. Table 19 contains the performance of the considered methods in terms of satisfying the applied constraints. SQP, PSO-LDIW, and PSO-NLIW failed to meet all applied constraints and thus was unable to solve the design problem successfully. Both PSO-RIW and PSO-CIW provide higher randomness to the momentum of a particle which helped in satisfying all the applied constraints.

5.3. Comparison of convergence performance

The statistical performance of UAPSO shows its superiority to the compared variants; however, to determine whether UAPSO managed to avoid premature convergence, the convergence curves of all considered variants, including UAPSO, are studied in Fig. 19. The convergence curves display the historical

**Fig. 19.** Convergence curves of UAPSO and the considered variants for the three-bar truss design problem variants.**Fig. 20.** Comparison of the mean function value and distribution of results obtained by UAPSO (with 50% less FEs) with the considered PSO variants.

performance of each variant's best-obtained solution throughout the 100 runs. From Fig. 19, it can be determined that the six

Table 20

Statistical performance of UAPSO with varying function evaluations.

Best	Worst	Mean	Standard deviation	FEs
21.440613	21.679870	21.542426	4.93 E-02	20,000
21.441485	21.740747	21.581358	0.10	18,000
21.442610	21.740747	21.586674	0.11	16,000
21.443804	21.740747	21.601395	0.10	14,000
21.450910	21.977204	21.640044	0.13	12,000
21.463146	21.977347	21.668766	0.12	10,000

variants performed in a similarly by converging in the last 1000 iterations. The robustness of an optimization method is revealed by its ability in avoiding premature convergence, and this can be seen in the zoomed-in section of Fig. 19. The zoomed-in section shows that the variants have, all, gone through multiple phases where particles were stuck in local minima and that apart from UAPSO, the other variants converged to their final solution in the last 500 iterations. In contrary, UAPSO kept on discovering better solutions until converging in the final 50 iterations, and this is despite UAPSO's slower start, compared to the other methods, as evidenced by its weaker performance in the initial 1000 iterations. This shows the robustness of the proposed method and coupled with its strong statistical performance, UAPSO proved to be successful in navigating the particles through the search space compared to the other variants. Both UAPSO and PSO-CIW allows the particles to cover a broader range of momentum, which allowed them to outperform the other variants. In PSO-CIW, particles navigate according to a random chaotic mapping, whereas in UAPSO particles navigate according to the evolutionary state. However, due to the evolutionary state, UAPSO avoids stagnation unlike PSO-CIW and the others (see Fig. 19).

5.4. Performance of UAPSO with varying function evaluations

UAPSO managed to outperform the considered variants with an equal number of function evaluations. Table 20 shows the statistical performance of UAPSO with varying function evaluations. The number of function evaluations was reduced by 50% from 20,000 FEs to 10,000 FEs. Here, UAPSO was run 100 times to extract statistical data. Naturally, the performance of UAPSO was affected negatively with the reduction of the function evaluations but comparing the performance of UAPSO at lower FEs with the considered PSO variants reveals the effectiveness of UAPSO. The best performing PSO variant (excluding UAPSO) was PSO-CIW; UAPSO with 20% less FEs managed to perform better across all statistical measures as can be seen in Table 21. The consistent and efficient behavior of UAPSO was demonstrated at lower FEs as UAPSO managed to produce a lower worst and mean value, even with 50% less FEs, in comparison with the considered variants as can be seen in Fig. 20. The results show that on average UAPSO with 50% less FEs designed a three-bar truss that weighed between 7% to 12% less in structural weight compared to the considered PSO.

6. Conclusion

Due to the Particle Swarm Optimization method's slow convergence and sensitivity to the initial control settings, a self-adaptive mechanism, known as the Unique Adaptive Particle Swarm Optimization (UAPSO), was proposed to promote performance by adapting the particles through a feedback system. Furthermore, in UAPSO, constraints are satisfied by the preservation of feasible solutions and the penalization of unfeasible ones. A set of constrained optimization problems were considered to validate the performance and effectiveness of the proposed

method. The numerical experiments illustrated that the proposed method managed to have the quickest convergence speed, in comparison with the considered methods, in all but one of the studied problems, and also performed efficiently by consistently producing solutions close to the global optimum. Based on the results of this paper, UAPSO has shown that it can be a practical approach in solving constrained problems as it managed to overcome premature convergence and promote convergence speed.

In the future, a fundamental study can be conducted to understand the trajectory of the particles, which would help in the introduction of useful information to the feedback system. The interaction of a particle with its neighbor directly affects its search information and space. Therefore, different swarm learning mechanisms need to be studied to determine the optimal topology. Furthermore, the type of constraint handling technique employed has an impact on the overall performance of the algorithm and thus will need to be considered as well.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.asoc.2019.105653>.

Appendix A. Benchmark constrained optimization problems

A.1. Constrained optimization function – g01

$$\min f(x) = (x_1 - 2)^2 + (x_2 - 1)^2$$

$$c(x)_1 = x_1 - 2x_2 + 1 = 0$$

$$c(x)_2 = \frac{x_1^2}{4} - x_2^2 + 1 \leq 0$$

$$\text{where } -10 \leq x_i \leq 10 (i = 1, 2).$$

A.2. Constrained optimization function – g02

$$\min f(x) = x_1^2 + (x_2 - 1)^2$$

$$c(x)_1 = x_2 - x_1^2 = 0$$

$$\text{where } -1 \leq x_i \leq 1 (i = 1, 2).$$

A.3. Constrained optimization function – g03

$$\min f(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

$$c(x)_1 = 4.84 - (x_1 - 0.05)^2 - (x_2 - 2.5)^2 \geq 0$$

$$c(x)_2 = x_1^2 + (x_2 - 2.5)^2 - 4.84 \geq 0$$

$$\text{where } 0 \leq x_i \leq 6 (i = 1, 2).$$

A.4. Constrained optimization function – g04

$$\min f(x) = (x_1 - 10)^2 + 5(x_2 - 12)^2 + x_3^4 + 3(x_4 - 11)^2 + 10x_5^6 + 7x_6^2 + x_7^4 - 4x_6x_7 - 10x_6 - 8x_7$$

$$c(x)_1 = 127 - 2x_1^2 - 3x_2^4 - x_3 - 4x_4^2 - 5x_5 \geq 0$$

$$c(x)_2 = 282 - 7x_1 - 3x_3 - 10x_3^2 - x_4 + x_5 \geq 0$$

$$c(x)_3 = 196 - 23x_1 - x_2^2 - 6x_6^2 + 8x_7 \geq 0$$

$$c(x)_4 = -4x_1^2 - x_2^2 + 3x_1x_2 - 2x_3^2 - 5x_6 + 11x_7 \leq 0$$

$$\text{where } -10 \leq x_i \leq 10 (i = 1, 2, \dots, 7).$$

Table 21

Comparison between PSO-CIW with 20,000 FEs and UAPSO with 16,000 FEs.

Variant	Best	Worst	Mean	Standard deviation
PSO-CIW (with 20,000 FEs)	21.4429	25.2582	22.2640	1.00
UAPSO (with 16,000 FEs)	21.4426	21.7407	21.5867	0.11

A.5. Speed reducer

$$\min f(x) = 0.7854x_1x_2^2(3.3333x_3^2 + 14.9334x_3 - 43.0934) - 1.508x_1(x_6^2 + x_7^2) + 7.4777(x_6^3 + x_7^3) + 0.7854(x_4x_6^2 + x_5x_7^2)$$

$$c(x)_1 = \frac{27}{x_1x_2^2x_3} - 1 \leq 0$$

$$c(x)_2 = \frac{397.5}{x_1x_2^2x_3^2} - 1 \leq 0$$

$$c(x)_3 = \frac{1.93x_4^3}{x_2x_6^4x_3} - 1 \leq 0$$

$$c(x)_4 = \frac{1.93x_5^3}{x_2x_6^4x_3} - 1 \leq 0$$

$$c(x)_5 = \frac{\sqrt{745\left(\frac{x_4}{x_2x_3}\right)^2 + 16.9 \times 10^6}}{110x_6^3} - 1 \leq 0$$

$$c(x)_6 = \frac{\sqrt{745\left(\frac{x_5}{x_2x_3}\right)^2 + 157.5 \times 10^6}}{85x_7^3} - 1 \leq 0$$

$$c(x)_7 = \frac{x_2x_3}{40} - 1 \leq 0$$

$$c(x)_8 = \frac{5x_2}{x_1} - 1 \leq 0$$

$$c(x)_9 = \frac{x_1}{12x_2} - 1 \leq 0$$

$$c(x)_{10} = \frac{1.5x_6 + 1.9}{x_4} - 1 \leq 0$$

$$c(x)_{11} = \frac{1.1x_7 + 1.9}{x_5} - 1 \leq 0$$

where $2.6 \leq x_1 \leq 3.6$, $0.7 \leq x_2 \leq 0.8$, $17 \leq x_3 \leq 28$, $7.3 \leq x_4 \leq 8.3$, $7.3 \leq x_5 \leq 8.3$, $2.9 \leq x_6 \leq 3.9$ and $5.0 \leq x_7 \leq 5.5$.

A.6. Pressure vessel

$$\min f(x) = 0.6224x_1x_3x_4 + 1.7781x_2x_3^2 + 3.1661x_1^2x_4 + 19.84x_1^2x_3$$

$$c(x)_1 = -x_1 + 0.0193x_3 \leq 0$$

$$c(x)_2 = -x_2 + 0.00954x_3 \leq 0$$

$$c(x)_3 = \pi x_3^2x_4 - \left(\frac{4\pi x_3^3}{3}\right) + 1.296(10^6) \leq 0$$

$$c(x)_4 = x_4 - 240 \leq 0$$

where $0 \leq x_i \leq 100$ ($i = 1, 2$) and $10 \leq x_i \leq 200$ ($i = 3, 4$).

A.7. Tension/compression spring

$$\min f(x) = (x_3 + 2)x_2x_1^2$$

$$c(x)_1 = 1 - \frac{x_2^3x_3}{71785x_1^4} \leq 0$$

$$c(x)_2 = \frac{4x_2^2 - x_1x_2}{12566(x_2x_1^3 - x_1^4)} + \frac{1}{5108x_1^2} - 1 \leq 0$$

$$c(x)_3 = 1 - \frac{140.45x_1}{x_2^2x_3} \leq 0$$

$$c(x)_4 = \frac{x_2 + x_1}{1.5} - 1 \leq 0$$

where $0.05 \leq x_1 \leq 2.00$, $0.25 \leq x_2 \leq 1.30$ and $2.00 \leq x_3 \leq 15.00$.

A.8. Welded beam

$$\min f(x) = 1.10471x_2x_1^2 + 0.04811x_3x_4(14.0 + x_2)$$

$$c(x)_1 = \tau(x) - \tau_{\max} \leq 0$$

$$c(x)_2 = \sigma(x) - \sigma_{\max} \leq 0$$

$$c(x)_3 = x_1 - x_4 \leq 0$$

$$c(x)_4 = 0.10471x_1^2 + 0.04811x_3x_4(14.0 + x_2) - 5 \leq 0$$

$$c(x)_5 = 0.125 - x_1 \leq 0$$

$$c(x)_6 = \delta(x) - \delta_{\max} \leq 0$$

$$c(x)_7 = P - P_c(x) \leq 0$$

where the other parameters are defined as:

$$\tau(x) = \sqrt{(\tau')^2 + 2\tau'\tau''\left(\frac{x_2}{2R}\right) + (\tau'')^2},$$

$$\tau' = \frac{P}{\sqrt{2}x_1x_2}, \quad \tau'' = \frac{MR}{J}$$

$$M = P\left(L + \frac{x_2}{2}\right), \quad R = \sqrt{\frac{x_2^2}{4} + \left(\frac{x_1 + x_3}{2}\right)^2},$$

$$J = \frac{2x_1x_2}{\sqrt{2}}\left(\frac{x_2^2}{12} + \left(\frac{x_1 + x_3}{2}\right)^2\right)$$

$$\sigma(x) = \frac{6PL}{x_3^2x_4}, \quad \delta(x) = \frac{4PL^3}{Ex_3^3x_4},$$

$$P_c(x) = \frac{4.013\sqrt{EG\left(\frac{x_2^2x_3^6}{36}\right)}}{L^2}\left(1 - \frac{x_3}{2L}\sqrt{\frac{E}{4G}}\right)$$

Also, the value of the parameters are set as thus:

$$P = 6000 \text{ lb}, \quad L = 14 \text{ in}, \quad E = 30 \times 10^6 \text{ psi}, \quad G = 30 \times 10^6 \text{ psi}$$

$$\tau_{\max} = 13600 \text{ psi}, \quad \sigma_{\max} = 30000 \text{ psi}, \quad \delta_{\max} = 0.25 \text{ in}$$

where $0.1 \leq x_i \leq 2$ ($i = 1, 4$) and $0.1 \leq x_i \leq 10$ ($i = 2, 3$).

Appendix B. Real-world structural optimization problem – three-bar truss

$$\min f(x) = W = \rho g \sum_{i=1}^3 a_i \sqrt{x_i^2 + 0.5^2}$$

$$c(x)_1 = \left| \frac{E(ux_i + 0.5v_i)}{x_i^2 + 0.5^2} \right| - S_a \leq 0 \quad \text{for } i = 1, 2, 3$$

$$c(x)_2 = Ea_i \frac{ux_i + 0.5v}{\sqrt{x_i^2 + 0.5^2}} - \frac{\pi Ea_i}{4(x_i^2 + 0.5^2)} \leq 0 \quad \text{for } i = 1, 2, 3$$

$$c(x)_3 = -a_i + 3.2 \times 10^{-6} \leq 0 \quad \text{for } i = 1, 2, 3$$

$$c(x)_4 = x_1 - x_2 + 0.1 \leq 0$$

$$c(x)_5 = x_2 - x_3 + 0.1 \leq 0$$

$$c(x)_6 = \left| \sqrt{u^2 + v^2} \right| \leq \delta$$

$$c(x)_7 = -\min(\omega) + \omega_0 \leq 0$$

where $-1 \leq x_i \leq 1 (i = 1, 2, 3)$ and $-1 \leq a_i \leq 1 (i = 1, 2, 3)$.

References

- [1] J.S. Arora, *Introduction To Optimum Design*, Academic Press, 2017.
- [2] J.H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, Cambridge, MA, 1992.
- [3] M. Dorigo, V. Maniezzo, A. Colnari, The ant system: optimization by a colony of cooperating agents, *IEEE Trans. Syst. Man Cybern. B* 26 (1) (1996) 29–41.
- [4] X.S. Yang, *Nature-Inspired Optimization Algorithms*, Elsevier, London, 2014.
- [5] S. Mirjalili, Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm, *Knowl.-Based Syst.* 89 (2015) 228–249.
- [6] X.-S. Yang, A.H. Gandomi, Bat algorithm: a novel approach for global engineering optimization, *Eng. Comput.* 29 (5) (2012) 464–483.
- [7] X. Meng, Y. Liu, X. Gao, H. Zhang, A new bio-inspired algorithm: chicken swarm optimization, *Advances in Swarm Intelligence*, Springer International Publishing, Frome, England, 2014, pp. 80–98.
- [8] X.-S. Yang, S. Deb, Cuckoo search via Lévy flights, in: *World Congress on Nature & Biologically Inspired Computing (NaBIC 2009)*, 2009, Coimbatore, India, pp. 210–214.
- [9] M. Yazdani, F. Jolai, Lion optimization algorithm (LOA): A nature-inspired metaheuristic algorithm, *J. Comput. Design Eng.* 3 (1) (2016) 24–36.
- [10] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *IEEE International Conference on Neural Networks*, Perth, WA, Australia, 1995, pp. 1942–1948.
- [11] S. Panda, N.P. Padhy, Comparison of particle swarm optimization and genetic algorithm for FACTS-based controller design, *Appl. Soft Comput.* 8 (4) (2008) 1418–1427.
- [12] M.A. Panduro, C.A. Brizuela, L.I. Balderas, D.A. Acosta, A comparison of genetic algorithms, particle swarm optimization and the differential evolution method for the design of scannable circular antenna arrays, *Prog. Electromagn. Res. B* 13 (2009) 171–186.
- [13] V.G. Gudise, G.K. Venayagamoorthy, Comparison of particle swarm optimization and backpropagation as training algorithms for neural networks, in: *Proceedings of the IEEE Swarm Intelligence Symposium*, Indianapolis, IN, USA, 2003, pp. 110–117.
- [14] Y. Wang, J. Lv, L. Zhu, Y. Ma, Crystal structure prediction via particle swarm optimization, *Phys. Rev. B* 82 (9) (2010) 094116–1–094116–8.
- [15] S. Pandey, L. Wu, S.M. Guru, R. Buyya, A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments, in: *IEEE International Conference on Advanced Information Networking and Applications*, Perth, WA, Australia, 2010, pp. 400–407.
- [16] X. Hu, R.C. Eberhart, Y. Shi, Engineering optimization with particle swarm, in: *Proceedings of the IEEE Swarm Intelligence Symposium*, Indianapolis, IN, USA, 2003, pp. 53–57.
- [17] F. van den Bergh, A. Engelbrecht, A new locally convergent particle swarm optimizer, in: *Proceedings of IEEE International Conference on Systems, Man and Cybernetics*, Yasmine Hammamet, Tunisia, 2002, pp. 96–101.
- [18] A. Carlisle, G. Dozier, An Off-The-Shelf PSO, in: *Proceeding of Workshop on Particle Swarm Optimization*, Indianapolis, IN, USA, 2001, pp. 1–6.
- [19] Y. Shi, R. Eberhart, A modified particle swarm optimizer, in: *IEEE International Conference on Evolutionary Computation Proceedings*, in: *IEEE World Congress on Computational Intelligence*, Anchorage, AK, USA, 1998, pp. 69–73.
- [20] T. Beielstein, K.E. Parsopoulos, M.N. Vrahatis, Tuning PSO parameters through sensitivity analysis, in: *Interne Bericht Des Sonderforschungsbereichs (SFB) 531 Computational Intelligence No.CI-124/02*, Universität Dortmund, 2002.
- [21] G. Ciuprina, D. Ioan, I. Munteanu, Use of intelligent-particle swarm optimization in electromagnetics, *IEEE Trans. Magn.* 38 (2) (2002) 1037–1040.
- [22] J.J. Liang, A.K. Qin, S. Baskar, P.N. Suganthan, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, *IEEE Trans. Evol. Comput.* 10 (3) (2006) 281–295.
- [23] X. Hu, R. Eberhart, Multiobjective optimization using dynamic neighborhood particle swarm optimization, in: *Proceedings of the Congress on Evolutionary Computation*, Honolulu, HI, USA, 2002, pp. 1667–1681.
- [24] C.A. Coello Coello, G.T. Pulido, M.S. Lechuga, Handling multiple objectives with particle swarm optimization, *IEEE Trans. Evol. Comput.* 8 (3) (2004) 256–279.
- [25] J. Kennedy, R.C. Eberhart, A discrete binary version of the particle swarm algorithm, in: *IEEE International Conference on Systems, Man, and Cybernetics*, Orlando, FL, USA, 1997, pp. 4104–4109.
- [26] E.C. Laskari, K.E. Parsopoulos, M.N. Vrahatis, Particle swarm optimization for integer programming, in: *Proceedings of the Congress on Evolutionary Computation*, Honolulu, HI, USA, 2002, pp. 1576–1581.
- [27] A. Nickabadi, M.M. Ebadzadeh, R. Safabakhsh, A novel particle swarm optimization algorithm with adaptive inertia weight, *Appl. Soft Comput.* 11 (4) (2011) 3658–3670.
- [28] M.D. Isiet, *Self-adapting control parameters in particle swarm optimization*, MASC. Thesis, University of British Columbia, 2019.
- [29] J. Kennedy, The behavior of particles, in: *International Conference on Evolutionary Programming*, San Diego, CA, USA, 1998, pp. 581–589.
- [30] M. Clerc, The swarm and the queen: towards a deterministic and adaptive particle swarm optimization, in: *Proceedings of the Congress on Evolutionary Computation*, Washington, DC, USA, 1999, pp. 1951–1957.
- [31] R. Mendes, J. Kennedy, J. Neves, The fully informed particle swarm: simpler, maybe better, *IEEE Trans. Evol. Comput.* 8 (3) (2004) 204–210.
- [32] A. Ratnaweera, S.K. Halgamuge, H.C. Watson, Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients, *IEEE Trans. Evol. Comput.* 8 (3) (2004) 240–255.
- [33] Q. He, L. Wang, An effective co-evolutionary particle swarm optimization for constrained engineering design problems, *Eng. Appl. Artif. Intell.* 20 (1) (2007) 89–99.
- [34] Y. Jiang, T. Hu, C.C. Huang, X. Wu, An improved particle swarm optimization algorithm, *Appl. Math. Comput.* 193 (1) (2007) 231–239.
- [35] Q. He, L. Wang, A hybrid particle swarm optimization with a feasibility-based rule for constrained optimization, *Appl. Math. Comput.* 186 (2) (2007) 1407–1422.
- [36] E.F. Campana, G. Liuzzi, S. Lucidi, D. Peri, V. Piccialli, A. Pinto, New global optimization methods for ship design problems, *Opt. Eng.* 10 (2009) 533–555.
- [37] A. Serani, C. Leotardi, U. Iemma, E.F. Campana, G. Fasano, M. Diez, Parameter selection in synchronous and asynchronous deterministic particle swarm optimization for ship hydrodynamics problems, *Appl. Soft Comput.* 49 (2016) 313–334.
- [38] R. Pellegrini, A. Serani, C. Leotardi, U. Iemma, E.F. Campana, M. Diez, Formulation and parameter selection of multi-objective deterministic particle swarm for simulation-based optimization, *Appl. Soft Comput.* 58 (2017) 714–731.
- [39] H. Liu, Z. Cai, Y. Wang, Hybridizing particle swarm optimization with differential evolution for constrained numerical and engineering optimization, *Appl. Soft Comput.* 10 (2) (2010) 629–640.
- [40] L. dos Santos Coelho, Gaussian quantum-behaved particle swarm optimization approaches for constrained engineering design problems, *Expert Syst. Appl.* 37 (2010) 1676–1683.
- [41] Z.-H. Zhan, J. Zhang, Y. Li, Y.-H. Shi, Orthogonal learning particle swarm optimization, *IEEE Trans. Evol. Comput.* 15 (6) (2011) 832–847.
- [42] W.-N. Chen, J. Zhang, Y. Lin, N. Chen, Z.-H. Zhan, H.S.-H. Chung, Y. Li, Y.-H. Shi, Particle swarm optimization with an aging leader and challengers, *IEEE Trans. Evol. Comput.* 17 (2) (2013) 241–258.
- [43] R. Cheng, Y. Jin, A social learning particle swarm optimization algorithm for scalable optimization, *Inform. Sci.* 291 (2015) 43–60.
- [44] N. Lynn, P.N. Suganthan, Ensemble particle swarm optimizer, *Appl. Soft Comput.* 55 (2017) 533–548.
- [45] K. Chen, F. Zhou, Y. Wang, L. Yin, An ameliorated particle swarm optimizer for solving numerical optimization problems, *Appl. Soft Comput.* 73 (2018) 482–496.
- [46] M. He, M. Liu, R. Wang, X. Jiang, B. Liu, H. Zhou, Particle swarm optimization with damping factor and cooperative mechanism, *Appl. Soft Comput.* 76 (2019) 45–52.
- [47] W. Ye, W. Feng, S. Fan, A novel multi-swarm particle swarm optimization with dynamic learning strategy, *Appl. Soft Comput.* 61 (2017) 832–843.
- [48] Y. Feng, G.-F. Teng, A.-X. Wang, Y.-M. Yao, Chaotic inertia weight in particle swarm optimization, in: *International Conference on Innovative Computing, Information and Control*, Kumamoto, Japan, 2007, pp. 475–479.
- [49] Z.-H. Zhan, J. Zhang, Y. Li, H.S.-H. Chung, Adaptive particle swarm optimization, *IEEE Trans. Syst. Man Cybern. B* 39 (6) (2009) 1362–1381.
- [50] E. Mezura-Montes, C.A. Coello Coello, Constraint-handling in nature-inspired numerical optimization: past, present and future, *Swarm Evol. Comput.* 1 (4) (2011) 173–194.
- [51] C.A. Coello Coello, Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art, *Comput. Methods Appl. Mech. Engrg.* 191 (11–12) (2002) 1245–1287.

- [52] K. Parsopoulos, M. Vrahatis, Unified Particle Swarm Optimization for Solving Constrained Engineering Optimization Problems, in: *Lecture Notes in Computer Science (LNCS)*, 3612, 2005, pp. 582–591.
- [53] K.E. Parsopoulos, M.N. Vrahatis, Particle swarm optimization method for constrained optimization problems, *Intell. Technol. - Theory Appl. New Trends Intell. Technol.* 76 (2002) 214–220.
- [54] X. Hu, R. Eberhart, Solving constrained nonlinear optimization problems, in: *Proceedings of the World Multiconference on Systemics, Cybernetics and Informatics*, Orlando, FL, USA, 2002, pp. 203–206.
- [55] R. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: *Proceedings of the International Symposium on Micro Machine and Human Science*, Nagoya, Japan, 1995, pp. 39–43.
- [56] M.R. Bonyadi, Z. Michalewicz, Particle swarm optimization for single objective continuous space problems: a review, *Evol. Comput.* 25 (1) (2017) 1–54.
- [57] A.P. Engelbrecht, *Particle Swarm Optimization*, in *Computational Intelligence: An Introduction*, John Wiley & Sons, Hoboken, NJ, 2007, pp. 289–358.
- [58] R. Poli, J. Kennedy, T. Blackwell, Particle swarm optimisation: an overview, *Swarm Intell.* 1 (1) (2007) 33–57.
- [59] R.C. Eberhart, Y. Shi, Comparing inertia weights and constriction factors in particle swarm optimization, in: *Proceedings of the Congress on Evolutionary Computation*, La Jolla, CA, USA, 2000, pp. 84–88.
- [60] A.P. Engelbrecht, *Computational Intelligence: An Introduction*, second ed., Wiley, 2007.
- [61] R.C. Eberhart, Y. Shi, Tracking and optimizing dynamic systems with particle swarms, in: *Proceedings of the Congress on Evolutionary Computation*, Seoul, South Korea, 2001, pp. 94–97.
- [62] Y. Shi, R.C. Eberhart, Fuzzy adaptive particle swarm optimization, in: *Proceedings of the Congress on Evolutionary Computation*, Seoul, South Korea, 2001, pp. 101–106.
- [63] A. Chatterjee, P. Siarry, Nonlinear inertia weight variation for dynamic adaptation in particle swarm optimization, *Comput. Oper. Res.* 33 (3) (2006) 859–871.
- [64] M. Taherkhani, R. Safabakhsh, A novel stability-based adaptive inertia weight for particle swarm optimization, *Appl. Soft Comput.* 38 (2016) 281–295.
- [65] P.N. Suganthan, Particle swarm optimiser with neighbourhood operator, in: *Proceedings of the Congress on Evolutionary Computation*, Washington, DC, USA, 1991, pp. 1958–1962.
- [66] J. Kennedy, The particle swarm: social adaptation of knowledge, in: *Proceedings of IEEE International Conference on Evolutionary Computation*, Indianapolis, IN, USA, 1997, pp. 303–308.
- [67] X. Yang, J. Yuan, J. Yuan, H. Mao, A modified particle swarm optimizer with dynamic adaptation, *Appl. Math. Comput.* 189 (2) (2007) 1205–1213.
- [68] M. Tanweer, S. Suresh, N. Sundararajan, Self regulating particle swarm optimization algorithm, *Inform. Sci.* 294 (2015) 182–202.
- [69] M.S. Arumugam, M.V. Rao, On the improved performances of the particle swarm optimization algorithms with adaptive parameters, cross-over operators and root mean square (rms) variants for computing optimal control of a class of hybrid systems, *Appl. Soft Comput.* 8 (1) (2008) 324–336.
- [70] F. Van den Bergh, A. Engelbrecht, A study of particle swarm optimization particle trajectories, *Inform. Sci.* 176 (8) (2006) 937–971.
- [71] L. dos Santos Coelho, Gaussian quantum-behaved particle swarm optimization approaches for constrained engineering design problems, *Expert Syst. Appl.* 37 (2010) 1676–1683.
- [72] I.C. Trelea, The particle swarm optimization algorithm: convergence analysis and parameter selection, *Inform. Process. Lett.* 85 (6) (2003) 317–325.
- [73] F. van den Bergh, A.P. Engelbrecht, A cooperative approach to particle swarm optimization, *IEEE Trans. Evol. Comput.* 8 (3) (2004) 225–239.
- [74] R.C. Eberhart, Y. Shi, Particle swarm optimization: developments, applications and resources, in: *Proceedings of the Congress on Evolutionary Computation*, Seoul, South Korea, 2001, pp. 81–86.
- [75] E. Mezura-Montes, C.A. Coello Coello, An empirical study about the usefulness of evolution strategies to solve constrained optimization problems, *Int. J. Gen. Syst.* 37 (4) (2008) 443–473.
- [76] Z. Michalewicz, M. Schoenauer, Evolutionary algorithms for constrained parameter optimization problems, *Evol. Comput.* 4 (1) (1996) 1–32.
- [77] J.T. Richardson, M.R. Palmer, G.E. Liepins, M. Hilliard, Some guidelines for genetic algorithms with penalty functions, in: *Proceedings of the Third International Conference on Genetic Algorithms*, San Mateo, CA, USA, 1989, pp. 41–49.
- [78] S. Vakili, *Analysis of water colling process of steel strups on runout table*, Ph.D. Thesis, University of British Columbia, 2011.
- [79] M.R. Alrasheed, C.W. de Silva, M.S. Gadala, A modified particle swarm optimization scheme and its application in electronic heat sink design, in: *ASME/Pacific Rim Technical Conference and Exhibition on Packaging and Integration of Electronic and Photonic Systems, MEMS, and NEMS*, Vancouver, BC, Canada, 2007.
- [80] K.H. Lee, S.W. Baek, K.W. Kim, Inverse radiation analysis using repulsive particle swarm optimization algorithm, *Int. J. Heat Mass Transfer* 51 (11–12) (2008) 2772–2783.
- [81] R.V. O'Neill, R.H. Gardner, J.B. Mankin, Analysis of parameter error in a nonlinear model, *Ecol. Model.* 8 (1980) 297–311.
- [82] D.M. Hamby, A review of techniques for parameter sensitivity analysis of environmental models, *Environ. Monit. Assess.* 32 (2) (1994) 135–154.
- [83] H. Wang, Z. Hu, Y. Sun, Q. Su, X. Xia, Modified backtracking search optimization algorithm inspired by simulated annealing for constrained engineering optimization problems, *Comput. Intell. Neurosci.* 2018 (2018) 1–27.
- [84] A. Sadollah, A. Bahreininejad, H. Eskandar, M. Hamdi, Mine blast algorithm: a new population based algorithm for solving constrained engineering optimization problems, *Appl. Soft Comput.* 13 (5) (2013) 2592–2612.
- [85] L. Wang, L.-p. Li, An effective differential evolution with level comparison for constrained engineering design, *Struct. Multidiscip. Optim.* 41 (6) (2010) 947–963.
- [86] B. Akay, D. Karaboga, Artificial bee colony algorithm for large-scale problems and engineering design optimization, *J. Intell. Manuf.* 23 (4) (2012) 1001–1014.
- [87] Y. Wang, Z. Cai, Y. Zhou, Z. Fan, Constrained optimization based on hybrid evolutionary algorithm and adaptive constraint-handling technique, *Struct. Multidiscip. Optim.* 37 (4) (2009) 395–413.
- [88] E. Mezura-Montes, C.A. Coello Coello, J. Velázquez-Reyes, Increasing successful offspring and diversity in differential evolution for engineering design, in: *proceedings of the Seventh International Conference on Adaptive Computing in Design and Manufacture (ACDM '06)*, 2006, pp. 131–139.
- [89] C. Zhang, Q. Lin, L. Gao, X. Li, Backtracking search algorithm with three constraint handling methods for constrained optimization problems, *Expert Syst. Appl.* 42 (21) (2015) 7831–7845.
- [90] F.Z. Huang, L. Wang, Q. He, An effective co-evolutionary differential evolution for constrained optimization, *Appl. Math. Comput.* 186 (1) (2007) 340–356.
- [91] C.A. Coello Coello, E. Mezura-Montes, Constraint-handling in genetic algorithms through the use of dominance-based tournament selection, *Adv. Eng. Inform.* 16 (3) (2002) 193–203.
- [92] R. Oftadeh, M. Mahjoob, M. Shariatpanahi, A novel meta-heuristic optimization algorithm inspired by group hunting of animals: Hunting search, *Comput. Math. Appl.* 60 (7) (2010) 2087–2098.
- [93] K. Seok Lee, Z. Woo Geem, A new meta-heuristic algorithm for continuous engineering optimization: harmony search theory and practice, *Comput. Methods Appl. Mech. Engrg.* 194 (2005) 3902–3933.
- [94] M. Zhang, W. Luo, X. Wang, Differential evolution with dynamic stochastic selection for constrained optimization, *Inform. Sci.* 178 (2008) 3043–3074.
- [95] A. Baykasoglu, S. Akpinar, Weighted superposition attraction (WSA): a swarm intelligence algorithm for optimization problems – part 2: constrained optimization, *Appl. Soft Comput.* 37 (2015) 396–415.
- [96] A. Askarzadeh, A novel metaheuristic method for solving constrained engineering optimization problems: Crow search algorithm, *Comput. Struct.* 169 (2016) 1–12.
- [97] A. Kaveh, A. Dadras, A novel meta-heuristic optimization algorithm: Thermal exchange optimization, *Adv. Eng. Softw.* 110 (2017) 69–84.
- [98] J. Golinski, An adaptive optimization system applied to machine synthesis, *Mech. Mach. Theory* 8 (4) (1973) 419–436.
- [99] P.E. Onate Yumbra, J.M. Ramirez, C.A. Coello Coello, Optimal power flow subject to security constraints solved with a particle swarm optimizer, *IEEE Trans. Power Syst.* 23 (1) (2008) 33–40.
- [100] E. Sandgren, Nonlinear integer and discrete programming in mechanical design optimization, *J. Mech. Des.* 112 (2) (1990) 223–229.
- [101] J.S. Arora, *Introduction To Optimum Design*, McGraw Hill, New York, 1989.
- [102] S.S. Rao, *Engineering Optimization*, Wiley, New Delhi, 1996.
- [103] N.D. Lagaros, M. Papadrakakis, G. Kokossalakis, Structural optimization using evolutionary algorithms, *Comput. Struct.* 80 (7–8) (2002) 571–589.
- [104] S. L.A., *Structural design by systematic synthesis*, in: *Conference on Electronic Computation*, Pittsburgh, PA, USA, 1960, pp. 105–132.
- [105] F.L. Stasa, *Applied Finite Element Analysis for Engineers*, Holt, Rinehart and Winston, New York, N.Y., 1985.
- [106] D. Halliday, R. Resnick, J. Walker, *Fundamentals of Physics Extended*, Wiley, New York, 2013.
- [107] P.T. Boggs, J.W. Tolle, Sequential quadratic programming for large-scale nonlinear optimization, *J. Comput. Appl. Math.* 124 (1–2) (2000) 123–137.



Mewael Isiet is a Ph.D. student of Mining Engineering at the University of British Columbia in Vancouver, Canada. Previously, Mewael Isiet completed his MASc of Mechanical Engineering, under the supervision of Dr. Mohamed Gadala, at the University of British Columbia in 2019. His current research interests include design optimization, mechanistic model development, sensitivity analysis and mineral processing. mewael_d@alumni.ubc.ca



Dr. Gadala is a Professor Emeritus of Mechanical Engineering at the University of British Columbia in Vancouver, Canada. Also, Dr. Gadala is the Chair of Mechanical Engineering Department at Abu Dhabi University in UAE. His current research interests include finite element and numerical simulation of structural & CFD problems; online monitoring of rotating equipment & turbo machinery; inverse heat transfer analysis for cooling on runout table, fracture mechanics and design optimization. Dr. Gadala is the recipient of the NSERC University-Industry Synergy award; Patrick Campbell chair in design in Mechanical Engineering at UBC for 12 years; Established PACE (Partner for Advancement of CAE Education) lab in UBC with multi million dollars industrial contributions of hardware & software licenses. Prof. Gadala has authored & co-authored over 180 refereed papers, 4 book chapters, 4 patents and 4 software manuals. Before joining academia, Prof. Gadala has worked in various industries in the US and Canada for more than twelve years and he also taught at the Univ. of Michigan-Dearborn. mohamed.gadala@adu.ac.ae and gadala@mech.ubc.ca